Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 06, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your lab environment. That is where the learning happens.

মূল্যায়ন ও উত্তর নির্দেশিকা

মূল্যায়ন, উত্তর নির্দেশিকা ও কুইক রেফারেন্স — Docker & Container Engineering

এই অংশটি **DevOps Engineering Mastery — Volume 06: Docker & Container Engineering** বইয়ের মূল্যায়ন ও সহায়ক অংশ। এখানে আছে ১২০টি বহুনির্বাচনি প্রশ্ন (MCQ), ৩০টি সংক্ষিপ্ত প্রশ্ন, ২০টি পরিস্থিতিভিত্তিক সমস্যা-সমাধান প্রশ্ন, ১০টি নিরাপত্তা ও প্রোডাকশন ইনসিডেন্ট কেস স্টাডি, একটি ব্যবহারিক চূড়ান্ত পরীক্ষা, সম্পূর্ণ উত্তর নির্দেশিকা, ক্যাপস্টোন মূল্যায়ন রুব্রিক, ১৮০টি পদের শব্দকোষ (Glossary), Docker CLI, Dockerfile ও Compose রেফারেন্স, নেটওয়ার্কিং ও স্টোরেজ রেফারেন্স, চেকলিস্ট, ট্রাবলশুটিং ডিসিশন ট্রি, দক্ষতা-ট্র্যাকিং শিট এবং অফিসিয়াল রেফারেন্স। চলমান উদাহরণ প্রতিষ্ঠান **TechNova** এবং MERN স্ট্যাক (MongoDB, Express, React, Next.js, Node.js) প্রসঙ্গ এই পুরো অংশে ব্যবহৃত হয়েছে।

> **সতর্কতা — আগে চেষ্টা, তারপর উত্তর:** উত্তর নির্দেশিকা পড়ার আগে প্রতিটি প্রশ্ন নিজে চেষ্টা করুন। MCQ-এর অক্ষর মুখস্থ করলে কিছু শিখবেন না — ব্যাখ্যাটি বুঝুন, তারপর নিজের কনটেইনারে প্রয়োগ করুন। চূড়ান্ত পরীক্ষাটি সময় ধরে, নোট বন্ধ রেখে দেওয়ার চেষ্টা করুন।

> **কীভাবে ব্যবহার করবেন:** প্রতিটি অধ্যায় পড়ার পর সংশ্লিষ্ট MCQ ব্লকটি সমাধান করুন। ভুল হওয়া প্রশ্নের অধ্যায়টি আবার পড়ুন এবং ল্যাব ওয়ার্কবুকের সংশ্লিষ্ট ল্যাবটি করুন। ক্যাপস্টোন শুরু করার আগে রুব্রিকটি ভালো করে পড়ুন — সেখানেই বলা আছে কোন মানদণ্ডে কত নম্বর।

মূল্যায়ন কাঠামো এক নজরে

অংশ	পরিমাণ	উদ্দেশ্য
MCQ (অংশ A)	১২০টি	২৪টি অধ্যায়ের ধারণাগত বোঝাপড়া যাচাই
সংক্ষিপ্ত প্রশ্ন (অংশ B)	৩০টি	সংক্ষেপে ব্যাখ্যা করার ক্ষমতা

পরিস্থিতিভিত্তিক প্রশ্ন (অংশ C)	২০টি	বাস্তব সমস্যায় নির্ণয় ও সিদ্ধান্ত
ইনসিডেন্ট কেস স্টাডি (অংশ D)	১০টি	নিরাপত্তা ও প্রোডাকশন ঘটনা থেকে শিক্ষা
চূড়ান্ত পরীক্ষা (অংশ E)	১০০ নম্বর, ৩ ঘণ্টা	তত্ত্ব + কনফিগারেশন + হাতে-কলমে কাজ + ইনসিডেন্ট রেসপন্স
উত্তর নির্দেশিকা (অংশ F)	সব প্রশ্নের উত্তর ও ব্যাখ্যা	স্ব-মূল্যায়ন ও ক্যাপস্টোন রুব্রিক
শব্দকোষ (অংশ G)	১৮০টি পদ	বাংলা ব্যাখ্যাসহ প্রযুক্তিগত শব্দভাণ্ডার
কুইক রেফারেন্স (অংশ H–K)	রেফারেন্স, চেকলিস্ট, ডিসিশন ট্রি, ট্র্যাকিং শিট	কাজের পাশে রাখার সহায়ক

নিয়ম: এই বইয়ের সব উদাহরণে কাল্পনিক ডোমেইন (`technova.example`), TEST-NET আইপি (`203.0.113.x`) এবং কাল্পনিক প্লেসহোল্ডার (`CHANGE-ME-...`) ব্যবহার করা হয়েছে। কোনো আসল secret, token, পাসওয়ার্ড বা মূল্য এই বইয়ে নেই। সব নমুনা টার্মিনাল আউটপুট চিত্রণমূলক। TechNova-র সব সার্ভিস, হোস্টনেম ও ঠিকানা কাল্পনিক।

২৪টি অধ্যায়ের মানচিত্র

অধ্যায় ১ ভার্চুয়াল মেশিন বনাম কনটেইনার; ২ Linux namespace; ৩ cgroups ও resource; ৪ Docker আর্কিটেকচার; ৫ ইমেজ ও লেয়ার; ৬ Dockerfile ভিত্তি; ৭ build cache ও context; ৮ multi-stage build; ৯ registry; ১০ নেটওয়ার্কিং ভিত্তি; ১১ DNS ও service discovery; ১২ ভলিউম ও স্টোরেজ; ১৩ Compose ভিত্তি; ১৪ MERN অ্যাপ কনটেইনারাইজেশন; ১৫ environment ও secrets; ১৬ কনটেইনার নিরাপত্তা; ১৭ resource limit ও performance; ১৮ লগিং ও observability; ১৯ প্রোডাকশন অপারেশন; ২০ আপডেট, rollback ও deployment কৌশল; ২১ Docker in CI/CD; ২২ Kubernetes-পূর্ব ধারণা ও orchestration প্রসঙ্গ; ২৩ ক্লাউড কনটেইনার সার্ভিস (AWS/Azure/GCP/IBM Cloud); ২৪ ক্যাপস্টোন — TechNova MERN প্ল্যাটফর্মের সম্পূর্ণ কনটেইনারাইজেশন।

অংশ A — বহুনির্বাচনি প্রশ্ন

১২০টি বহুনির্বাচনি প্রশ্ন (MCQ)

প্রতিটি প্রশ্নের চারটি বিকল্প আছে — ক, খ, গ, ঘ। সঠিক উত্তরটি বেছে নিন। উত্তর ও ব্যাখ্যা আছে অংশ F-এ, তবে আগে নিজে চেষ্টা করুন। প্রশ্নগুলো শুধু মুখস্থ পরীক্ষা করে না — কেন একটি সিদ্ধান্ত নেওয়া হয়, কোনো ব্যর্থতার কারণ কী, এবং কোন পদ্ধতি কখন উপযুক্ত, তা যাচাই করে।

ব্লক ১ — প্রশ্ন ১–২০: VM বনাম কনটেইনার, namespace, cgroups (অধ্যায় ১–৩)

1. VM আর কনটেইনারের মূল পার্থক্য কী?
   ক) VM নিজের kernel চালায়, কনটেইনার host kernel ভাগ করে নেয় খ) কনটেইনার নিজের kernel চালায় গ) কোনো পার্থক্য নেই ঘ) VM-এ কোনো OS লাগে না
2. একই হোস্টে ৫০টি কনটেইনার চালানো VM-এর চেয়ে সাধারণত সহজ কেন?
   ক) কনটেইনারে RAM লাগে না খ) প্রতিটি কনটেইনার হালকা, পূর্ণ OS boot করে না, kernel ভাগ করে গ) কনটেইনারে CPU লাগে না ঘ) কনটেইনার ডিস্ক ব্যবহার করে না
3. TechNova-র একটি Node.js API এক VM-এ চলে, আরেকটি কনটেইনারে। কোনটি দ্রুত শুরু হবে এবং কেন?
   ক) VM, কারণ kernel আলাদা খ) কনটেইনার, কারণ শুধু প্রসেস শুরু হয়, OS boot হয় না গ) দুটোই সমান ঘ) VM, কারণ image ছোট
4. PID namespace-এর কাজ কী?
   ক) নেটওয়ার্ক আলাদা করা খ) কনটেইনারের ভেতরে প্রসেসগুলোকে আলাদা PID দৃষ্টিভঙ্গি দেওয়া, হোস্টের প্রসেস লুকানো গ) ফাইল মুছে ফেলা ঘ) CPU সীমিত করা
5. Mount namespace ছাড়া কনটেইনার কী করতে পারত না?
   ক) নিজের আলাদা filesystem দৃষ্টিভঙ্গি (যেমন নিজের `/`) রাখতে পারত না খ) নেটওয়ার্ক ব্যবহার করতে পারত না গ) প্রসেস চালাতে পারত না ঘ) লগ লিখতে পারত না
6. Network namespace কী আলাদা করে?
   ক) শুধু হোস্টনেম খ) ইন্টারফেস, IP, রাউটিং টেবিল ও পোর্টের নিজস্ব দৃষ্টিভঙ্গি গ) শুধু DNS ঘ) শুধু ফায়ারওয়াল নিয়ম
7. UTS namespace কী নিয়ন্ত্রণ করে?
   ক) CPU সীমা খ) হোস্টনেম ও ডোমেইন নামের দৃষ্টিভঙ্গি গ) মেমরি সীমা ঘ) ব্যবহারকারীর তালিকা
8. User namespace-এর নিরাপত্তা-সুবিধা কী?
   ক) কনটেইনারের root হোস্টে সাধারণ ব্যবহারকারী হিসেবে ম্যাপ হতে পারে, ফলে হোস্টে root ক্ষমতা পায় না খ) কনটেইনার দ্রুত হয় গ) image ছোট হয় ঘ) নেটওয়ার্ক বন্ধ হয়
9. cgroups-এর মূল কাজ কী?
   ক) প্রসেস আলাদা করা খ) CPU, মেমরি, I/O ও PID-এর মতো সম্পদের ব্যবহার সীমিত ও হিসাব করা গ) ফাইল এনক্রিপ্ট করা ঘ) DNS দেওয়া
10. `--memory=512m` দিলে কনটেইনার ৫১২ MB ছাড়ালে কী হয়?
    ক) কিছুই হয় না খ) কনটেইনার ধীর হয় কিন্তু চলে গ) kernel OOM killer প্রসেসটি মেরে ফেলতে পারে, কনটেইনার থেমে যায় ঘ) হোস্ট রিস্টার্ট হয়

11. `--cpus=1.5`-এর অর্থ কী?
    ক) ১.৫টি CPU কোর একচেটিয়াভাবে বরাদ্দ খ) মোট ১.৫ কোর-সমপরিমাণ CPU সময় ব্যবহারের ঊর্ধ্বসীমা গ) ১৫% CPU ঘ) ১.৫ GHz গতি
12. একটি কনটেইনার হোস্টের সব CPU খেয়ে ফেলছে। প্রথমে কোন সীমা দেবেন?
    ক) `--memory` খ) `--cpus` বা `--cpu-shares` দিয়ে CPU সীমা গ) `--read-only` ঘ) `--hostname`
13. Namespace isolation থাকলেও kernel ভাগ করার ঝুঁকি কী?
    ক) কোনো ঝুঁকি নেই খ) kernel দুর্বলতা থাকলে এক কনটেইনার থেকে হোস্ট বা অন্য কনটেইনারে প্রভাব পড়তে পারে গ) image বড় হয় ঘ) নেটওয়ার্ক ধীর হয়
14. VM-এ strong isolation দরকার এমন কাজে কনটেইনারের বদলে VM বেছে নেওয়া কখন যুক্তিযুক্ত?
    ক) কখনোই নয় খ) অবিশ্বস্ত কোড চালাতে হলে বা আলাদা kernel দরকার হলে গ) শুধু ছোট অ্যাপে ঘ) শুধু ডেভেলপমেন্টে
15. `unshare --pid --fork` দিয়ে কী বোঝা যায়?
    ক) নতুন namespace তৈরি করে প্রসেস আলাদা করার ধারণা খ) Docker ইনস্টল করা গ) image তৈরি করা ঘ) ভলিউম মাউন্ট করা
16. একটি কনটেইনারে `ps aux` চালালে হোস্টের সব প্রসেস দেখা গেলে কী বোঝায়?
    ক) PID namespace ঠিক আছে খ) কনটেইনারটি host PID namespace ভাগ করছে (`--pid=host`) বা isolation ভাঙা গ) স্বাভাবিক ঘটনা ঘ) image-এর সমস্যা
17. cgroups v2-এ `memory.max` আর `cpu.max` কী করে?
    ক) শুধু তথ্য দেখায় খ) যথাক্রমে মেমরি ও CPU ব্যবহারের কঠোর ঊর্ধ্বসীমা নির্ধারণ করে গ) ফাইল মুছে ফেলে ঘ) নেটওয়ার্ক বন্ধ করে
18. TechNova-র MongoDB কনটেইনার মেমরি খেয়ে হোস্ট ধীর করছে। টেকসই সমাধান কোনটি?
    ক) কনটেইনার মুছে ফেলা খ) `--memory` ও `--memory-swap` সীমা দেওয়া এবং MongoDB-র cache সেটিং পর্যালোচনা করা গ) হোস্টের RAM খুলে ফেলা ঘ) লগ বন্ধ করা
19. IPC namespace কী আলাদা করে?
    ক) shared memory ও semaphore-এর মতো আন্তঃপ্রসেস যোগাযোগের মাধ্যম খ) নেটওয়ার্ক পোর্ট গ) ফাইলসিস্টেম ঘ) ব্যবহারকারী
20. কনটেইনারকে “হালকা VM” বলা ভুল কেন?
    ক) কনটেইনার আসলে ভারী খ) কনটেইনার VM নয় — এটি kernel-স্তরের isolation-সহ একটি প্রসেস-সমষ্টি, নিজের kernel নেই গ) VM-এর চেয়ে ধীর ঘ) কনটেইনারে OS থাকে না একদম

ব্লক ২ — প্রশ্ন ২১–৪০: Docker আর্কিটেকচার, ইমেজ ও লেয়ার (অধ্যায় ৪–৫)

1. Docker daemon (`dockerd`)-এর কাজ কী?
    ক) শুধু image ডাউনলোড করা খ) কনটেইনার, ইমেজ, নেটওয়ার্ক ও ভলিউম পরিচালনা করা — API শুনে containerd-কে নির্দেশ দেওয়া গ) শুধু লগ দেখানো ঘ) কোড লেখা
2. Docker client আর daemon কীভাবে কথা বলে?
    ক) শুধু ইমেইলে খ) REST API-র মাধ্যমে, সাধারণত Unix socket (`/var/run/docker.sock`) বা TCP-তে গ) শুধু SSH-এ ঘ) ফাইল ভাগ করে

3. `docker.sock` কনটেইনারে মাউন্ট করা কেন বিপজ্জনক?
   ক) কনটেইনার ধীর হয় খ) ওই কনটেইনার হোস্টের Docker daemon নিয়ন্ত্রণ করতে পারে — কার্যত হোস্টে root ক্ষমতা পায় গ) image মুছে যায় ঘ) নেটওয়ার্ক বন্ধ হয়
4. containerd-এর ভূমিকা কী?
   ক) শুধু registry খ) কনটেইনারের lifecycle (তৈরি, চালানো, থামানো) পরিচালনা করা — Docker daemon-এর নিচের runtime স্তর গ) কোড কম্পাইল করা ঘ) DNS দেওয়া
5. runc কী করে?
   ক) image তৈরি করে খ) OCI স্পেসিফিকেশন অনুযায়ী আসল কনটেইনার প্রসেস শুরু করে (namespace ও cgroups প্রয়োগ করে) গ) নেটওয়ার্ক রাউট করে ঘ) লগ সংরক্ষণ করে
6. OCI (Open Container Initiative) কেন গুরুত্বপূর্ণ?
   ক) এটি একটি কোম্পানি খ) ইমেজ ফরম্যাট ও runtime-এর উন্মুক্ত মান নির্ধারণ করে, ফলে টুলগুলো পরস্পরের সঙ্গে কাজ করে গ) এটি শুধু Docker-এর জন্য ঘ) এটি একটি registry
7. Docker image আসলে কী?
   ক) একটি চলমান প্রসেস খ) read-only লেয়ারের স্তরীভূত সংগ্রহ ও metadata — যা থেকে কনটেইনার তৈরি হয় গ) একটি VM ডিস্ক ঘ) একটি ডেটাবেস
8. Image layer কীভাবে তৈরি হয়?
   ক) প্রতিটি Dockerfile নির্দেশনা একটি নতুন read-only লেয়ার যোগ করে (কিছু নির্দেশনা ছাড়া) খ) প্রতিটি `docker run`-এ গ) প্রতিটি লগ লাইনে ঘ) এলোমেলোভাবে
9. Layer caching কেন কাজ করে?
   ক) প্রতিটি লেয়ারের content-addressable hash থাকে; একই নির্দেশনা ও ইনপুট হলে একই লেয়ার পুনঃব্যবহৃত হয় খ) Docker এলোমেলোভাবে সিদ্ধান্ত নেয় গ) ফাইলের নাম দেখে ঘ) সময় দেখে
10. একটি লেয়ার বদলালে তার উপরের সব লেয়ার আবার তৈরি হয় কেন?
    ক) Docker-এর বাগ খ) প্রতিটি লেয়ার নিচের লেয়ারের উপর নির্ভর করে; নিচের hash বদলালে উপরের cache অকার্যকর হয় গ) registry বাধ্য করে ঘ) নেটওয়ার্কের কারণে
11. `docker image ls`-এ একই image-এর একাধিক tag থাকলে কী হয়?
    ক) একাধিক কপি ডিস্কে থাকে খ) একই লেয়ার-সমষ্টিকে একাধিক নামে চেনা যায়, ডিস্কে একটিই কপি থাকে গ) image মুছে যায় ঘ) কনটেইনার দ্বিগুণ হয়
12. কনটেইনারের writable layer কোথায় থাকে?
    ক) image-এর ভেতরে খ) image লেয়ারগুলোর উপরে, কনটেইনার-নির্দিষ্ট পাতলা writable স্তরে গ) registry-তে ঘ) হোস্টের RAM-এ শুধু
13. কনটেইনার মুছে ফেললে তার writable layer-এর ডেটা কী হয়?
    ক) থেকে যায় খ) সাধারণত মুছে যায়, যদি না volume বা bind mount-এ রাখা হয় গ) image-এ যোগ হয় ঘ) registry-তে যায়
14. `docker commit` কেন প্রোডাকশনে নিরুৎসাহিত করা হয়?
    ক) এটি কাজ করে না খ) এতে Dockerfile ছাড়া হাতে বদলানো অবস্থা image হয় — পুনরাবৃত্তিযোগ্য নয়, কী বদলেছে তা জানা যায় না গ) এটি ধীর ঘ) এটি image বড় করে না
15. Base image হিসেবে `alpine` বেছে নেওয়ার সুবিধা কী?
    ক) সব প্যাকেজ থাকে খ) খুব ছোট, আক্রমণ-ক্ষেত্র (attack surface) কম, pull ও scan দ্রুত হয় গ) সব অ্যাপ চলে ঘ) glibc সবসময় থাকে

16. `node:20-alpine` বনাম `node:20`-এর পার্থক্য কী?
    ক) একই খ) alpine সংস্করণ musl libc-ভিত্তিক, ছোট; পূর্ণ সংস্করণ Debian-ভিত্তিক, বড় কিন্তু বেশি টুল ও glibc সামঞ্জস্য গ) alpine-এ Node থাকে না ঘ) পূর্ণ সংস্করণে npm থাকে না
17. Image digest ব্যবহার করলে কী নিশ্চয়তা পাওয়া যায়?
    ক) image ছোট হবে খ) ঠিক একই content-এর image-ই টানা হবে, tag বদলে গেলেও গ) image দ্রুত চলবে ঘ) registry বিনামূল্যে হবে
18. `docker image prune -a` কী মুছে ফেলে?
    ক) শুধু চলমান কনটেইনার খ) ব্যবহৃত হচ্ছে না এমন সব image (tag ছাড়া dangling-সহ) গ) সব ভলিউম ঘ) সব নেটওয়ার্ক
19. TechNova-র image ১.২ GB — প্রথম সন্দেহ কোথায়?
    ক) tag-এর নাম খ) base image বড়, build টুল চূড়ান্ত image-এ রয়ে গেছে, বা cache/অপ্রয়োজনীয় ফাইল কপি হয়েছে গ) registry ধীর ঘ) DNS সমস্যা
20. Union filesystem (যেমন overlay2) কী করে?
    ক) একাধিক read-only লেয়ারকে একটিমাত্র filesystem দৃষ্টিভঙ্গিতে জোড়া দেয় খ) ফাইল এনক্রিপ্ট করে গ) নেটওয়ার্ক রাউট করে ঘ) প্রসেস আলাদা করে

ব্লক ৩ — প্রশ্ন ৪১–৬০: Dockerfile, build cache, multi-stage (অধ্যায় ৬–৮)

1. Dockerfile-এ নির্দেশনার ক্রম কেন গুরুত্বপূর্ণ?
    ক) শুধু দেখতে ভালো লাগে খ) কম বদলানো ধাপ আগে রাখলে cache বেশি কাজ করে, build দ্রুত হয় গ) ক্রমের কোনো প্রভাব নেই ঘ) Docker ক্রম উপেক্ষা করে
2. `COPY package*.json ./` আগে করে `npm ci` চালানো কেন ভালো অভ্যাস?
    ক) কোড ছোট হয় খ) সোর্স কোড বদলালেও dependency লেয়ার cache থেকে পুনঃব্যবহৃত হয় গ) npm দ্রুত হয় ঘ) image ছোট হয় না
3. `RUN npm install` বনাম `RUN npm ci`-এর পার্থক্য কী?
    ক) একই খ) `ci` lockfile অনুযায়ী ঠিক সংস্করণ ইনস্টল করে ও `node_modules` পরিষ্কার করে, পুনরাবৃত্তিযোগ্য build দেয় গ) `ci` শুধু dev-এ চলে ঘ) `install` দ্রুততর
4. `.dockerignore` ফাইলের কাজ কী?
    ক) image মুছে ফেলা খ) build context থেকে অপ্রয়োজনীয় ফাইল (যেমন `node_modules`, `.git`) বাদ দেওয়া, ফলে context ছোট ও build দ্রুত হয় গ) Dockerfile লুকানো ঘ) secret সংরক্ষণ করা
5. Build context খুব বড় হলে কী সমস্যা হয়?
    ক) কিছুই হয় না খ) প্রতিবার পুরো context daemon-এ পাঠাতে সময় ও ব্যান্ডউইথ খরচ হয় গ) image ছোট হয় ঘ) cache ভালো হয়
6. `WORKDIR /app` না দিলে কী হয়?
    ক) build ব্যর্থ হয় খ) পরের নির্দেশনাগুলো ডিফল্ট (`/`) ডিরেক্টরিতে চলে, পথ-সংক্রান্ত ভুল সহজ হয় গ) image মুছে যায় ঘ) কিছুই হয় না
7. `CMD` আর `ENTRYPOINT`-এর পার্থক্য কী?
    ক) একই খ) `ENTRYPOINT` কনটেইনারের মূল কমান্ড নির্ধারণ করে, `CMD` তার ডিফল্ট আর্গুমেন্ট দেয়; `docker run`-এর আর্গুমেন্ট `CMD`-কে বদলে দেয় গ) `CMD` শুধু build-এ চলে ঘ) `ENTRYPOINT` শুধু Compose-এ চলে

8. `CMD ["node", "server.js"]` (exec form) বনাম `CMD node server.js` (shell form)-এর পার্থক্য কী?
   ক) একই খ) exec form-এ PID 1 হিসেবে সরাসরি Node চলে ও signal পায়; shell form-এ shell PID 1 হয়, signal forwarding ভেঙে যেতে পারে গ) shell form দ্রুত ঘ) exec form-এ shell variable কাজ করে না একদম
9. একটি Node কনটেইনার `docker stop`-এ ১০ সেকেন্ড ধরে ঝুলে থাকে। সম্ভাব্য কারণ কী?
   ক) image বড় খ) PID 1 signal ধরছে না (shell form বা signal না-ধরা অ্যাপ), তাই Docker শেষে SIGKILL দেয় গ) DNS সমস্যা ঘ) ভলিউম পূর্ণ
10. `ENV NODE_ENV=production` Dockerfile-এ লেখার ঝুঁকি কী?
    ক) কোনো ঝুঁকি নেই খ) এটি image-এ স্থায়ী হয় ও সবাই দেখতে পারে; secret কখনো `ENV`-এ লেখা উচিত নয় গ) Node চলে না ঘ) build ব্যর্থ হয়
11. `ARG` আর `ENV`-এর পার্থক্য কী?
    ক) একই খ) `ARG` শুধু build সময়ে থাকে, `ENV` runtime-এও কনটেইনারে থাকে গ) `ARG` runtime-এ থাকে ঘ) `ENV` build-এ থাকে না
12. Multi-stage build-এর মূল সুবিধা কী?
    ক) build ধীর হয় খ) build টুল (যেমন TypeScript compiler) চূড়ান্ত image-এ না রেখে ছোট, নিরাপদ runtime image তৈরি করা যায় গ) Dockerfile ছোট হয় ঘ) cache বন্ধ হয়
13. Multi-stage-এ `COPY --from=builder /app/dist /app/dist` কী করে?
    ক) পুরো builder image কপি করে খ) শুধু প্রয়োজনীয় build ফলাফল (artifact) চূড়ান্ত stage-এ নেয় গ) builder মুছে ফেলে ঘ) registry-তে পাঠায়
14. BuildKit-এর `--mount=type=cache` কখন কাজে আসে?
    ক) secret লুকাতে খ) `npm ci`-এর cache পুনঃব্যবহার করে build দ্রুত করা, image-এ cache না রেখে গ) image ছোট করতে ঘ) নেটওয়ার্ক বন্ধ করতে
15. `RUN apt-get update && apt-get install -y ... && rm -rf /var/lib/apt/lists/*` এক লাইনে লেখা হয় কেন?
    ক) দেখতে ভালো খ) এক লেয়ারে ইনস্টল ও cache পরিষ্কার করলে image-এ apt cache থেকে যায় না গ) apt দ্রুত হয় ঘ) কোনো কারণ নেই
16. একই `RUN`-এ `apt-get update` আলাদা লেয়ারে লিখলে কী সমস্যা হয়?
    ক) cache-এর কারণে পুরনো package list ব্যবহার হতে পারে, ফলে ইনস্টল ব্যর্থ বা পুরনো সংস্করণ আসতে পারে খ) image ছোট হয় গ) build দ্রুত হয় ঘ) কিছুই হয় না
17. `USER node` নির্দেশনা কেন গুরুত্বপূর্ণ?
    ক) image ছোট হয় খ) কনটেইনার root হিসেবে না চলে অ-root ব্যবহারকারী হিসেবে চলে, নিরাপত্তা বাড়ে গ) build দ্রুত হয় ঘ) নেটওয়ার্ক ভালো হয়
18. `HEALTHCHECK` কী করে?
    ক) image scan করে খ) নিয়মিত কমান্ড চালিয়ে কনটেইনারটি আসলে সাড়া দিচ্ছে কি না যাচাই করে গ) লগ মুছে ফেলে ঘ) কনটেইনার রিস্টার্ট করে
19. `EXPOSE 3000` কি পোর্ট প্রকাশ করে?
    ক) হ্যাঁ, হোস্টে পোর্ট খুলে দেয় খ) না — এটি শুধু নথিভুক্তি/metadata; আসল প্রকাশ করতে `-p` বা Compose-এ `ports:` লাগে গ) হ্যাঁ, সবসময় ঘ) না, এটি build বন্ধ করে
20. TechNova-র Dockerfile-এ `COPY . .` প্রথম লাইনে আছে। সমস্যা কী?
    ক) কোনো সমস্যা নেই খ) যেকোনো ফাইল বদলালে dependency ইনস্টলসহ সব লেয়ার আবার তৈরি হয়, build ধীর হয় গ) image মুছে যায় ঘ) Node চলে না

ব্লক ৪ — প্রশ্ন ৬১–৮০: Registry, নেটওয়ার্কিং, DNS (অধ্যায় ৯–১১)

1. Container registry-র কাজ কী?
   ক) কনটেইনার চালানো খ) Docker image সংরক্ষণ, সংস্করণ চিহ্নিতকরণ ও বিতরণ করা গ) লগ সংরক্ষণ করা ঘ) কোড কম্পাইল করা
2. Public registry (Docker Hub) বনাম private registry — কখন private দরকার?
   ক) কখনোই নয় খ) TechNova-র নিজস্ব কোডের image বাইরে প্রকাশ করতে না চাইলে, অ্যাক্সেস নিয়ন্ত্রণ দরকার হলে গ) শুধু ছোট image-এর জন্য ঘ) শুধু dev-এ
3. `docker push technova/api:1.4.0` করার আগে কী দরকার?
   ক) কিছুই নয় খ) image-টি ওই নামে tag করা ও registry-তে authenticate করা (`docker login`) গ) কনটেইনার চালানো ঘ) ভলিউম মুছে ফেলা
4. Image tag `latest` প্রোডাকশনে ব্যবহারের ঝুঁকি কী?
   ক) কোনো ঝুঁকি নেই খ) `latest` পরিবর্তনশীল — কোন সংস্করণ চলছে তা নিশ্চিত করা যায় না, rollback অনিশ্চিত হয় গ) registry তা নেয় না ঘ) image মুছে যায়
5. Digest (`sha256:...`) দিয়ে pull করলে কী নিশ্চয়তা পাওয়া যায়?
   ক) image ছোট হবে খ) ঠিক একই content-এর image-ই আসবে গ) pull দ্রুত হবে ঘ) tag লাগবে না একদম
6. Registry-তে image scan (Trivy) কী খোঁজে?
   ক) কোডের স্টাইল খ) OS প্যাকেজ ও লাইব্রেরিতে জানা দুর্বলতা (CVE) গ) নেটওয়ার্ক সমস্যা ঘ) সার্ভারের গতি
7. Bridge network-এ দুটি কনটেইনার একে অপরকে কীভাবে চেনে?
   ক) শুধু IP দিয়ে খ) একই user-defined bridge network-এ থাকলে কনটেইনার নাম দিয়ে DNS resolution হয় গ) চেনা যায় না ঘ) শুধু হোস্টনেম দিয়ে
8. Default bridge network-এ কনটেইনার নামে DNS resolution হয় না কেন?
   ক) Docker-এর বাগ খ) legacy default bridge-এ embedded DNS নেই; user-defined network-এই সুবিধা থাকে গ) IP নেই বলে ঘ) পোর্ট বন্ধ বলে
9. `docker run -p 8080:3000 technova/api`-এর অর্থ কী?
   ক) কনটেইনারের ৩০০০ পোর্ট হোস্টের ৮০৮০ পোর্টে প্রকাশ করা খ) হোস্টের ৩০০০ পোর্ট কনটেইনারের ৮০৮০-এ যাওয়া গ) দুটি পোর্ট বন্ধ করা ঘ) নেটওয়ার্ক মুছে ফেলা
10. Host network (`--network host`) ব্যবহারের ঝুঁকি কী?
    ক) কোনো ঝুঁকি নেই খ) কনটেইনার হোস্টের নেটওয়ার্ক namespace ভাগ করে — isolation কমে, পোর্ট সংঘাত ও নিরাপত্তা ঝুঁকি বাড়ে গ) কনটেইনার ধীর হয় ঘ) DNS বন্ধ হয়
11. None network কখন ব্যবহার করা হয়?
    ক) সবসময় খ) কনটেইনারকে কোনো নেটওয়ার্ক না দিয়ে সম্পূর্ণ বিচ্ছিন্ন রাখতে চাইলে (যেমন একবারের batch কাজ) গ) ইন্টারনেট দ্রুত করতে ঘ) DNS ঠিক করতে
12. Overlay network কখন দরকার হয়?
    ক) এক হোস্টে খ) একাধিক হোস্টের (Swarm/Kubernetes) কনটেইনারগুলোকে একই virtual network-এ যুক্ত করতে গ) শুধু dev-এ ঘ) কখনোই নয়
13. TechNova-র API কনটেইনার MongoDB-কে `mongodb://mongo:27017` দিয়ে ডাকছে, কিন্তু `mongo` নামটি resolve হচ্ছে না। প্রথম যাচাই কী?
    ক) image মুছে ফেলা খ) দুটি কনটেইনার একই user-defined network-এ আছে কি না গ) RAM বাড়ানো ঘ) registry বদলানো

14. Embedded DNS (127.0.0.11) কী করে?
    ক) হোস্টের DNS বদলে দেয় খ) কনটেইনারের ভেতরে নাম resolution-এর অনুরোধ ধরে একই network-এর কনটেইনার নাম resolve করে গ) ইন্টারনেট বন্ধ করে ঘ) পোর্ট প্রকাশ করে
15. `--dns 8.8.8.8` দিলে কী হয়?
    ক) কনটেইনারের DNS সার্ভার বদলে যায় খ) হোস্টের DNS বদলে যায় গ) নেটওয়ার্ক বন্ধ হয় ঘ) কিছুই হয় না
16. একটি কনটেইনার ইন্টারনেটে যেতে পারছে না, কিন্তু অন্য কনটেইনার পারছে। সম্ভাব্য কারণ কী?
    ক) image বড় খ) কনটেইনারটি ভিন্ন network-এ, DNS ভুল, বা egress নিয়মে আটকে আছে গ) CPU কম ঘ) tag ভুল
17. `docker network inspect technova-net` দিয়ে কী জানা যায়?
    ক) image-এর লেয়ার খ) network-এর subnet, gateway, সংযুক্ত কনটেইনার ও IP বরাদ্দ গ) লগ ঘ) secret
18. Port conflict: হোস্টের ৩০০০ পোর্ট ইতিমধ্যে ব্যবহৃত। `-p 3000:3000` চালালে কী হবে?
    ক) দুটি কনটেইনার চলবে খ) bind ব্যর্থ হয়ে কনটেইনার শুরু হবে না — অন্য হোস্ট পোর্ট বেছে নিতে হবে গ) পুরনো কনটেইনার থেমে যাবে ঘ) কিছুই হবে না
19. Service discovery বলতে কী বোঝায়?
    ক) সার্ভিস মুছে ফেলা খ) চলমান সার্ভিসগুলো কীভাবে একে অপরকে খুঁজে পায় — নাম, DNS বা registry-র মাধ্যমে গ) লগ খোঁজা ঘ) image খোঁজা
20. Ingress (reverse proxy/Nginx) কনটেইনার-জগতে কেন ব্যবহার করা হয়?
    ক) ডেটাবেস চালাতে খ) বাইরের ট্রাফিক একটিমাত্র প্রবেশপথ দিয়ে গ্রহণ করে ভেতরের সার্ভিসগুলোতে রাউট করতে, TLS ও রাউটিং এক জায়গায় রাখতে গ) image তৈরি করতে ঘ) লগ মুছতে

ব্লক ৫ — প্রশ্ন ৮১–১০০: ভলিউম, Compose, MERN, secrets (অধ্যায় ১২–১৫)

1. Named volume বনাম bind mount-এর মূল পার্থক্য কী?
    ক) একই খ) named volume Docker পরিচালনা করে, হোস্ট পথ থেকে স্বাধীন; bind mount হোস্টের নির্দিষ্ট পথের সঙ্গে সরাসরি বাঁধা গ) bind mount-এ ডেটা থাকে না ঘ) volume-এ ফাইল লেখা যায় না
2. MongoDB-র ডেটা কনটেইনারের ভেতরে রাখলে কনটেইনার মুছলে কী হয়?
    ক) ডেটা থেকে যায় খ) ডেটা মুছে যায় — তাই volume ব্যবহার করা বাধ্যতামূলক গ) ডেটা image-এ যায় ঘ) ডেটা registry-তে যায়
3. `-v mongo-data:/data/db`-এর অর্থ কী?
    ক) হোস্টের `/data/db` মাউন্ট করা খ) `mongo-data` named volume-কে কনটেইনারের `/data/db`-তে মাউন্ট করা গ) ফাইল কপি করা ঘ) নেটওয়ার্ক তৈরি করা
4. Bind mount dev-এ কখন উপকারী?
    ক) প্রোডাকশন ডেটা রাখতে খ) হোস্টের সোর্স কোড কনটেইনারে মাউন্ট করে কোড বদলালেই (hot reload) ফল দেখতে গ) secret রাখতে ঘ) image ছোট করতে
5. Bind mount-এ হোস্টের ফাইলের permission সমস্যা হয় কেন?
    ক) UID/GID হোস্ট ও কনটেইনারে ভিন্ন হতে পারে; কনটেইনারের root লেখা ফাইল হোস্টে root-মালিকানাধীন হয় খ) Docker-এর বাগ গ) ফাইল বড় বলে ঘ) নেটওয়ার্কের কারণে
6. Volume-কে read-only মাউন্ট করা হয় কেন?
    ক) ডেটা দ্রুত করতে খ) কনটেইনার যাতে ভুল করে ডেটা বদলাতে না পারে, নিরাপত্তা ও অখণ্ডতা রক্ষা করতে গ) ডিস্ক বাঁচাতে ঘ) লগ বন্ধ করতে

7. `docker volume prune` চালানোর আগে কী যাচাই করবেন?
   ক) কিছুই নয় খ) কোনো ব্যবহৃত-না-হওয়া volume-এ দরকারি ডেটা আছে কি না — এটি স্থায়ীভাবে মুছে ফেলে গ) image-এর tag ঘ) DNS
8. Compose ফাইলে `services:`-এর কাজ কী?
   ক) নেটওয়ার্ক তৈরি করা খ) চালাতে হবে এমন প্রতিটি কনটেইনার (api, web, mongo) ঘোষণা করা গ) secret তৈরি করা ঘ) image push করা
9. Compose-এ `depends_on:` কী নিশ্চয়তা দেয়?
   ক) নির্ভরশীল সার্ভিস প্রস্তুত (ready) হয়েছে খ) শুধু শুরু করার ক্রম — সার্ভিসটি শুরু হয়েছে, কিন্তু প্রস্তুত কি না তা নিশ্চিত করে না গ) ডেটা প্রস্তুত ঘ) নেটওয়ার্ক প্রস্তুত
10. `depends_on`-এর সঙ্গে healthcheck জুড়লে কী লাভ হয়?
    ক) build দ্রুত হয় খ) `condition: service_healthy` দিয়ে নির্ভরশীল সার্ভিস সত্যিই সাড়া দেওয়ার পরেই পরের সার্ভিস শুরু হয় গ) image ছোট হয় ঘ) লগ বন্ধ হয়
11. Compose-এ `environment:` বনাম `env_file:`-এর পার্থক্য কী?
    ক) একই খ) `environment:` YAML-এ সরাসরি লেখা হয়, `env_file:` আলাদা ফাইল থেকে পড়ে — ফাইলটি Git-এ কমিট করা উচিত নয় গ) `env_file:` শুধু dev-এ চলে ঘ) `environment:` secret-এর জন্য নিরাপদ
12. TechNova-র API শুরু হয়েই MongoDB সংযোগে ব্যর্থ হচ্ছে। Compose-এ সবচেয়ে সম্ভাব্য কারণ কী?
    ক) image ভুল খ) API MongoDB প্রস্তুত হওয়ার আগেই শুরু হয়েছে — healthcheck-ভিত্তিক `depends_on` বা retry যুক্ত করা দরকার গ) CPU কম ঘ) tag ভুল
13. MERN অ্যাপে frontend (Next.js), API (Express) ও MongoDB — তিনটি কনটেইনারের মধ্যে যোগাযোগের সেরা উপায় কী?
    ক) সবাই host network ব্যবহার করবে খ) একই user-defined network-এ সার্ভিস নাম দিয়ে যোগাযোগ করবে গ) সবাই এক কনটেইনারে চলবে ঘ) IP hardcode করা
14. Next.js অ্যাপের build-time env বনাম runtime env-এর পার্থক্য কেন গুরুত্বপূর্ণ?
    ক) কোনো পার্থক্য নেই খ) `NEXT_PUBLIC_*` build সময়ে bundle-এ ঢুকে যায়; runtime secret কখনো build-time env-এ রাখা উচিত নয় গ) runtime env কাজ করে না ঘ) build-time env নিরাপদ
15. Secret কখনো Dockerfile-এ `ENV` বা image layer-এ রাখা উচিত নয় কেন?
    ক) image বড় হয় খ) image যার হাতে যাবে, সে-ই secret দেখতে পাবে — layer history-তেও থেকে যায় গ) build ব্যর্থ হয় ঘ) Node চলে না
16. Docker secrets (Swarm) বা runtime env file-এর সুবিধা কী?
    ক) secret Git-এ যায় খ) secret শুধু runtime-এ, মেমরিতে বা সীমিত ফাইলে দেওয়া হয়, image-এ থাকে না গ) secret দ্রুত হয় ঘ) image ছোট হয়
17. `docker compose up -d` বনাম `docker compose up`-এর পার্থক্য কী?
    ক) একই খ) `-d` detached mode — সার্ভিসগুলো পেছনে চলে, টার্মিনাল মুক্ত থাকে গ) `-d` build করে না ঘ) `-d` ভলিউম মুছে ফেলে
18. `docker compose down -v` কী করে?
    ক) শুধু কনটেইনার থামায় খ) কনটেইনার, নেটওয়ার্ক ও named volume-সহ সবকিছু মুছে ফেলে — ডেটা হারানোর ঝুঁকি আছে গ) শুধু image মুছে ফেলে ঘ) কিছুই করে না
19. Compose-এ `restart: unless-stopped` কখন উপযুক্ত?
    ক) dev-এ একবারের কাজে খ) প্রোডাকশন-ঘেঁষা সার্ভিসে — ব্যর্থ হলে বা হোস্ট রিস্টার্ট হলে সার্ভিস আবার চালু হবে গ) কখনোই নয় ঘ) শুধু ডেটাবেসে নয়

20. Compose ফাইলে obsolete `version:` key রাখলে কী হয়?
    ক) Compose ব্যর্থ হয় খ) আধুনিক Compose v2-এ এটি উপেক্ষা করা হয় ও সতর্কবার্তা আসতে পারে — সরিয়ে ফেলা উচিত গ) image মুছে যায় ঘ) নেটওয়ার্ক বন্ধ হয়

ব্লক ৬ — প্রশ্ন ১০১–১২০: নিরাপত্তা, resource, লগিং, প্রোডাকশন, ক্লাউড (অধ্যায় ১৬–২৪)

1. কনটেইনার root হিসেবে চালানো কেন ঝুঁকিপূর্ণ?
   ক) ধীর হয় খ) কনটেইনার থেকে বেরিয়ে আসা (escape) হলে হোস্টে root ক্ষমতা পাওয়ার সম্ভাবনা বাড়ে গ) image বড় হয় ঘ) লগ বন্ধ হয়
2. `--read-only` ফ্ল্যাগের সুবিধা কী?
   ক) কনটেইনার দ্রুত হয় খ) কনটেইনারের root filesystem লেখা যায় না — আক্রমণকারী ফাইল বদলাতে বা ম্যালওয়্যার রাখতে পারে না গ) ভলিউম মুছে যায় ঘ) নেটওয়ার্ক বন্ধ হয়
3. `--cap-drop=ALL` দিয়ে কী করা হয়?
   ক) সব capability বাদ দিয়ে শুধু প্রয়োজনীয় capability (যেমন `CHOWN`) যোগ করা — least privilege খ) কনটেইনার বন্ধ করা গ) CPU সীমা দেওয়া ঘ) লগ বন্ধ করা
4. `--privileged` কনটেইনার কেন প্রোডাকশনে নিষিদ্ধ প্রায়?
   ক) এটি ধীর খ) এটি হোস্টের প্রায় সব ডিভাইস ও kernel ক্ষমতা দেয় — কার্যত হোস্টে root গ) এটি image বড় করে ঘ) এটি DNS বন্ধ করে
5. Image-এ অপ্রয়োজনীয় প্যাকেজ রাখা কেন ঝুঁকি?
   ক) image সুন্দর হয় না খ) প্রতিটি প্যাকেজ সম্ভাব্য দুর্বলতা (CVE) ও আক্রমণ-ক্ষেত্র বাড়ায় গ) build দ্রুত হয় ঘ) লগ বাড়ে
6. Trivy বা Grype scan CI-তে fail করানো কখন যুক্তিযুক্ত?
   ক) কখনোই নয় খ) critical/high দুর্বলতা পেলে build আটকে দেওয়া — দুর্বল image প্রোডাকশনে যাওয়া রোধ করে গ) শুধু ছোট image-এ ঘ) শুধু dev-এ
7. OOMKilled (exit code 137) দেখলে প্রথম সন্দেহ কী?
   ক) DNS সমস্যা খ) কনটেইনার মেমরি সীমা ছাড়িয়েছে বা হোস্টে মেমরি শেষ হয়েছে গ) image ভুল ঘ) পোর্ট বন্ধ
8. `docker stats` দিয়ে কী দেখা যায়?
   ক) image-এর লেয়ার খ) চলমান কনটেইনারের CPU, মেমরি, নেটওয়ার্ক I/O ও block I/O-র live ব্যবহার গ) secret ঘ) লগের ইতিহাস
9. কনটেইনারের লগ `docker logs`-এ দেখতে হলে অ্যাপ কী করা উচিত?
   ক) ফাইলে লগ লেখা খ) stdout/stderr-এ লগ লেখা — Docker সেখান থেকেই লগ সংগ্রহ করে গ) ডেটাবেসে লগ লেখা ঘ) লগ বন্ধ করা
10. JSON-file logging driver-এর ঝুঁকি কী?
    ক) লগ দেখা যায় না খ) log rotation না করলে হোস্টের ডিস্ক লগে ভরে যেতে পারে গ) লগ ধীর হয় ঘ) লগ এনক্রিপ্ট হয় না
11. Healthcheck ব্যর্থ হলে orchestrator কী করতে পারে?
    ক) কিছুই নয় খ) কনটেইনারটি unhealthy চিহ্নিত করে রিস্টার্ট বা ট্রাফিক থেকে সরাতে পারে গ) image মুছে ফেলতে পারে ঘ) registry বন্ধ করতে পারে

12. Rolling update-এর সুবিধা কী?
    ক) ডাউনটাইম শূন্য হয় সবসময় খ) একবারে একটি/কয়েকটি কনটেইনার বদলে ধীরে ধীরে নতুন সংস্করণ আনা যায়, সমস্যা হলে থামানো যায় গ) ডেটা মুছে যায় ঘ) image ছোট হয়
13. Blue-green deployment-এ rollback কীভাবে সহজ হয়?
    ক) ডেটা মুছে ফেলা খ) পুরনো (blue) পরিবেশ চালু থাকে, ট্রাফিক ফিরিয়ে দিলেই rollback হয় গ) image আবার build করা ঘ) registry মুছে ফেলা
14. Canary deployment কখন উপযুক্ত?
    ক) সবসময় খ) নতুন সংস্করণ প্রথমে অল্প ট্রাফিকে পরীক্ষা করে মেট্রিক দেখে ধীরে ধীরে বাড়াতে চাইলে গ) ডেটা না থাকলে ঘ) test না থাকলে
15. CI/CD-তে Docker image build ও push কোন ধাপে হয়?
    ক) deploy-এর পরে খ) test পাসের পরে, deploy-এর আগে — একই artifact সব environment-এ ব্যবহৃত হয় গ) কোড লেখার আগে ঘ) লগ দেখার পরে
16. Kubernetes-এ যাওয়ার আগে Docker জ্ঞান কেন কাজে আসে?
    ক) আসে না খ) Pod, image, volume, network ও healthcheck ধারণাগুলো একই ভিত্তির উপর দাঁড়িয়ে আছে গ) Kubernetes Docker ব্যবহার করে না একদম ঘ) শুধু নাম একই
17. AWS ECS, Azure Container Instances বা GCP Cloud Run-এর সাধারণ সুবিধা কী?
    ক) সার্ভার পরিচালনা ছাড়াই কনটেইনার চালানো যায় (managed/serverless কনটেইনার সার্ভিস) খ) বিনামূল্যে সবকিছু গ) Docker শেখা লাগে না ঘ) ডেটা হারায় না কখনো
18. প্রোডাকশনে `latest` tag ও `:latest` image দিয়ে deploy করলে rollback-এ কী সমস্যা হয়?
    ক) কোনো সমস্যা নেই খ) কোন সংস্করণ চলছিল তা জানা যায় না, নির্দিষ্ট সংস্করণে ফেরা কঠিন হয় গ) deploy দ্রুত হয় ঘ) লগ বন্ধ হয়
19. TechNova-র প্রোডাকশন কনটেইনার রাতে থেমে গেল, সকালে কেউ জানল না। সবচেয়ে বড় ঘাটতি কী?
    ক) image বড় খ) healthcheck, restart policy, monitoring ও alerting-এর অভাব গ) tag ভুল ঘ) DNS ভুল
20. ক্যাপস্টোনে TechNova MERN প্ল্যাটফর্মের জন্য সবচেয়ে গুরুত্বপূর্ণ প্রমাণ (evidence) কী?
    ক) শুধু Dockerfile থাকা খ) multi-stage build, healthcheck, resource limit, non-root user, volume-এ ডেটা, Compose-এ পুরো স্ট্যাক চালু ও rollback পরিকল্পনা — সব একসাথে কাজ করা গ) শুধু image push করা ঘ) শুধু README লেখা

অংশ B — সংক্ষিপ্ত প্রশ্ন

৩০টি সংক্ষিপ্ত প্রশ্ন

প্রতিটি প্রশ্নের উত্তর ৩–৫ বাক্যে লিখুন। সংজ্ঞার পাশাপাশি একটি উদাহরণ বা TechNova প্রসঙ্গ দিন।

1. VM বনাম কনটেইনারের তিনটি মূল পার্থক্য লিখুন।
2. PID namespace ও mount namespace কীভাবে কনটেইনারকে আলাদা রাখে, ব্যাখ্যা করুন।
3. cgroups দিয়ে CPU ও মেমরি সীমা দেওয়ার প্রয়োজনীয়তা ব্যাখ্যা করুন।
4. Docker daemon, containerd ও runc-এর ভূমিকা আলাদা করে লিখুন।
5. Image layer ও কনটেইনারের writable layer-এর পার্থক্য ব্যাখ্যা করুন।
6. Layer cache কীভাবে কাজ করে এবং কোন নির্দেশনা বদলালে cache ভাঙে?
7. Dockerfile-এ নির্দেশনার ক্রম কীভাবে build সময়কে প্রভাবিত করে?
8. `.dockerignore` কেন ব্যবহার করবেন? দুটি উদাহরণ দিন।
9. Multi-stage build-এর সুবিধা একটি MERN উদাহরণ দিয়ে ব্যাখ্যা করুন।
10. `CMD` বনাম `ENTRYPOINT`-এর পার্থক্য ও ব্যবহার লিখুন।
11. Exec form বনাম shell form `CMD`-এর signal-সংক্রান্ত পার্থক্য ব্যাখ্যা করুন।
12. Registry-তে tag বনাম digest ব্যবহারের সুবিধা-অসুবিধা লিখুন।
13. User-defined bridge network কেন default bridge-এর চেয়ে ভালো?
14. কনটেইনার নাম দিয়ে DNS resolution কীভাবে কাজ করে?
15. `-p 8080:3000` বলতে কী বোঝায়? একটি উদাহরণ দিন।
16. Named volume, bind mount ও tmpfs mount-এর ব্যবহার আলাদা করে লিখুন।
17. Compose-এ `depends_on` কেন readiness-এর নিশ্চয়তা দেয় না? সমাধান কী?
18. Compose-এ healthcheck কীভাবে API-র প্রস্তুতি যাচাই করে?
19. Environment variable বনাম Docker secret — কখন কোনটি ব্যবহার করবেন?
20. Next.js-এ build-time ও runtime environment variable-এর পার্থক্য ব্যাখ্যা করুন।
21. Non-root user হিসেবে কনটেইনার চালানো কেন গুরুত্বপূর্ণ?
22. `--read-only` ও `--cap-drop=ALL` কীভাবে নিরাপত্তা বাড়ায়?
23. Image scanning (Trivy) CI-তে যুক্ত করার যুক্তি ব্যাখ্যা করুন।
24. OOMKilled (137) ও exit code 1-এর মধ্যে পার্থক্য কীভাবে নির্ণয় করবেন?
25. কনটেইনারের লগ stdout/stderr-এ লেখা কেন সেরা অভ্যাস?
26. JSON-file log driver-এ log rotation না করলে কী ঝুঁকি হয়?
27. Rolling, blue-green ও canary deployment-এর মধ্যে একটি পার্থক্য লিখুন।
28. CI/CD-তে একই image artifact সব environment-এ ব্যবহার করা কেন গুরুত্বপূর্ণ?
29. Healthcheck, restart policy ও monitoring — তিনটি একসাথে কেন দরকার?
30. TechNova-র MERN স্ট্যাককে তিনটি কনটেইনারে ভাগ করার সুবিধা ও একটি ঝুঁকি লিখুন।

অংশ C — পরিস্থিতিভিত্তিক প্রশ্ন

২০টি ট্রাবলশুটিং পরিস্থিতি (Scenarios)

প্রতিটি পরিস্থিতিতে অবস্থা ও উপসর্গ পড়ে প্রশ্নগুলোর উত্তর দিন: সম্ভাব্য কারণ, নির্ণয়ের ধাপ (কোন কমান্ড/লগ দেখবেন), সমাধান ও ভবিষ্যতে প্রতিরোধ।

পরিস্থিতি ১ — কনটেইনার শুরু হয়েই থেমে যাচ্ছে

অবস্থা: TechNova-র `api` কনটেইনার `docker compose up -d`-এর পর কয়েক সেকেন্ডে `Exited (1)` হচ্ছে। **উপসর্গ:** `docker logs api`-এ `Error: Cannot find module 'express'`। **প্রশ্ন:** (ক) সবচেয়ে সম্ভাব্য কারণ কী? (খ) কোন দুটি জিনিস প্রথমে যাচাই করবেন? (গ) স্থায়ী সমাধান কী?

পরিস্থিতি ২ — পোর্ট ইতিমধ্যে ব্যবহৃত

অবস্থা: `docker run -p 3000:3000 technova/api` চালালে `Bind for 0.0.0.0:3000 failed: port is already allocated`। **উপসর্গ:** হোস্টে আরেকটি প্রসেস ৩০০০ পোর্টে চলছে। **প্রশ্ন:** (ক) কোন কমান্ডে দখলকারী প্রসেসটি খুঁজবেন? (খ) দুটি নিরাপদ সমাধান কী? (গ) প্রোডাকশনে পোর্ট সংঘাত এড়াতে কী করবেন?

পরিস্থিতি ৩ — MongoDB-র সঙ্গে সংযোগ ব্যর্থ

অবস্থা: API কনটেইনার `MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017` দিচ্ছে। **উপসর্গ:** `localhost` ব্যবহার করা হয়েছে connection string-এ। **প্রশ্ন:** (ক) কেন `localhost` কনটেইনারের ভেতরে ভুল? (খ) সঠিক connection string কী হওয়া উচিত? (গ) MongoDB প্রস্তুত হওয়ার আগেই API শুরু হলে কী করবেন?

পরিস্থিতি ৪ — OOMKilled

অবস্থা: Next.js কনটেইনার মাঝে মাঝে থেমে যাচ্ছে, `docker ps -a`-এ `Exited (137)`। **উপসর্গ:** `docker inspect`-এ `OOMKilled: true`। **প্রশ্ন:** (ক) 137-এর অর্থ কী? (খ) `docker stats` ও লগে কী দেখবেন? (গ) সীমা বাড়ানো ছাড়া আর কী পর্যালোচনা করবেন?

পরিস্থিতি ৫ — DNS resolution ব্যর্থ

অবস্থা: API কনটেইনার থেকে `mongo` নামটি resolve হচ্ছে না, কিন্তু IP দিয়ে সংযোগ হয়। **উপসর্গ:** কনটেইনারগুলো default bridge-এ আছে। **প্রশ্ন:** (ক) মূল কারণ কী? (খ) সমাধান কী? (গ) ভবিষ্যতে নাম-ভিত্তিক যোগাযোগ কীভাবে নিশ্চিত করবেন?

পরিস্থিতি ৬ — Build cache বারবার ভাঙছে

অবস্থা: প্রতিবার ছোট কোড পরিবর্তনে পুরো `npm ci` আবার চলছে, build ৮ মিনিট নিচ্ছে। **উপসর্গ:** Dockerfile-এ প্রথমেই `COPY . .` আছে। **প্রশ্ন:** (ক) কেন cache ভাঙছে? (খ) Dockerfile কীভাবে সাজাবেন? (গ) `.dockerignore`-এর ভূমিকা কী?

পরিস্থিতি ৭ — Image অত্যন্ত বড়

অবস্থা: TechNova-র API image ১.৪ GB, pull করতে ৫ মিনিট লাগে। **উপসর্গ:** base image `node:20`, build টুল ও `node_modules`-এর dev dependency চূড়ান্ত image-এ আছে। **প্রশ্ন:** (ক) তিনটি বড় হওয়ার কারণ চিহ্নিত করুন। (খ) multi-stage দিয়ে কীভাবে ছোট করবেন? (গ) কীভাবে যাচাই করবেন কোন লেয়ার বড়?

পরিস্থিতি ৮ — ডেটা হারিয়ে গেছে

অবস্থা: `docker compose down -v` চালানোর পর MongoDB-র সব ডেটা নেই। **উপসর্গ:** কোনো ব্যাকআপ ছিল না। **প্রশ্ন:** (ক) কী ঘটেছে? (খ) এখন কী করা যায় (সৎ উত্তর দিন)? (গ) ভবিষ্যতে ডেটা রক্ষার তিনটি নিয়ম কী?

পরিস্থিতি ৯ — `docker stop` ঝুলে থাকছে

অবস্থা: API কনটেইনার `docker stop`-এ ১০ সেকেন্ড পর SIGKILL-এ থামছে। **উপসর্গ:** Dockerfile-এ shell form `CMD node server.js` আছে। **প্রশ্ন:** (ক) কেন graceful shutdown হচ্ছে না? (খ) সমাধান কী? (গ) Node অ্যাপে SIGTERM ধরার গুরুত্ব কী?

পরিস্থিতি ১০ — Registry push ব্যর্থ

অবস্থা: `docker push`-এ `denied: requested access to the resource is denied`। **উপসর্গ:** `docker login` করা হয়নি বা token-এর মেয়াদ শেষ। **প্রশ্ন:** (ক) সম্ভাব্য দুটি কারণ কী? (খ) নির্ণয়ের ধাপ কী? (গ) CI-তে credential কীভাবে নিরাপদে দেবেন?

পরিস্থিতি ১১ — Secret ফাঁস

অবস্থা: একজন ডেভেলপার `ENV MONGO_URI=mongodb://...` Dockerfile-এ কমিট করেছেন ও image push করেছেন। **উপসর্গ:** image history-তে secret দেখা যাচ্ছে। **প্রশ্ন:** (ক) কেন শুধু লাইন মুছে ফেলা যথেষ্ট নয়? (খ) এখন কী করতে হবে (rotate, revoke, rebuild)? (গ) ভবিষ্যতে secret কীভাবে দেবেন?

পরিস্থিতি ১২ — লগে ডিস্ক ভরে গেছে

অবস্থা: হোস্টের ডিস্ক ১০০% পূর্ণ, Docker daemon সাড়া দিচ্ছে না। **উপসর্গ:** `/var/lib/docker/containers/*/*-json.log` ফাইলগুলো বিশাল। **প্রশ্ন:** (ক) মূল কারণ কী? (খ) তাৎক্ষণিক নিরাপদ পদক্ষেপ কী? (গ) স্থায়ী log rotation কনফিগারেশন কী?

পরিস্থিতি ১৩ — কনটেইনার unhealthy, কিন্তু অ্যাপ চলছে

অবস্থা: `docker ps`-এ API `unhealthy`, কিন্তু curl করলে সাড়া দেয়। **উপসর্গ:** healthcheck `curl -f http://localhost:3000/health` ব্যর্থ হচ্ছে, কনটেইনারে `curl` ইনস্টল নেই। **প্রশ্ন:** (ক) কেন healthcheck ব্যর্থ হচ্ছে? (খ) দুটি সমাধান কী? (গ) healthcheck endpoint ডিজাইনের নিয়ম কী?

পরিস্থিতি ১৪ — CPU-তে প্রতিবেশী কনটেইনার ক্ষতিগ্রস্ত

অবস্থা: একটি ব্যাচ কনটেইনার সব CPU খেয়ে ফেলায় API ধীর হয়ে গেছে। **উপসর্গ:** কোনো resource limit দেওয়া হয়নি। **প্রশ্ন:** (ক) কেন এটি ঘটেছে? (খ) কোন limit দেবেন ও কেন? (গ) `--cpus` বনাম `--cpu-shares`-এর পার্থক্য কী?

পরিস্থিতি ১৫ — Compose-এ ভুল environment

অবস্থা: Staging-এ API প্রোডাকশন ডেটাবেসে সংযোগ করছে। **উপসর্গ:** `.env` ফাইলটি ভুল environment-এর, Compose-এ `env_file` একই ফাইল নির্দেশ করছে। **প্রশ্ন:** (ক) কী ভুল হয়েছে? (খ) environment আলাদা করার নিরাপদ পদ্ধতি কী? (গ) এই ভুলের প্রভাব সীমিত করতে কী করবেন?

পরিস্থিতি ১৬ — Image pull ধীর / ব্যর্থ

অবস্থা: নতুন হোস্টে `docker pull technova/api:1.4.0` বারবার timeout হচ্ছে। **উপসর্গ:** নেটওয়ার্ক ধীর, image বড়, কোনো mirror নেই। **প্রশ্ন:** (ক) সম্ভাব্য তিনটি কারণ কী? (খ) কীভাবে নির্ণয় করবেন? (গ) pull দ্রুত করার তিনটি উপায় কী?

পরিস্থিতি ১৭ — ভলিউম permission সমস্যা

অবস্থা: MongoDB কনটেইনার `Permission denied` দিয়ে শুরু হচ্ছে না। **উপসর্গ:** bind mount করা হোস্ট ডিরেক্টরির মালিকানা root-এর, কনটেইনার non-root ব্যবহারকারী হিসেবে চলছে। **প্রশ্ন:** (ক) কেন permission মিলছে না? (খ) দুটি সমাধান কী? (গ) named volume ব্যবহার করলে কেন সমস্যা কমে?

পরিস্থিতি ১৮ — আপডেটের পর অ্যাপ ভেঙে গেছে

অবস্থা: `latest` tag দিয়ে deploy করার পর API 500 দিচ্ছে। **উপসর্গ:** কোন সংস্করণ deploy হয়েছে তা কেউ জানে না। **প্রশ্ন:** (ক) কেন rollback কঠিন? (খ) তাৎক্ষণিক পদক্ষেপ কী? (গ) ভবিষ্যতে versioning ও rollback পরিকল্পনা কেমন হবে?

পরিস্থিতি ১৯ — কনটেইনার থেকে হোস্টে ফাইল লেখা যাচ্ছে

অবস্থা: একটি কনটেইনার হোস্টের `/etc` ডিরেক্টরিতে ফাইল লিখতে পারছে। **উপসর্গ:** `-v /:/host` দিয়ে চালানো হয়েছিল, সঙ্গে `--privileged`। **প্রশ্ন:** (ক) এই কনফিগারেশন কেন বিপজ্জনক? (খ) তাৎক্ষণিক পদক্ষেপ কী? (গ) ভবিষ্যতে কী নিয়ম মানবেন?

পরিস্থিতি ২০ — রাতে কনটেইনার থেমে গেছে, কেউ জানেনি

অবস্থা: প্রোডাকশন API রাত ২টায় থেমে গেছে, সকাল ৯টায় ব্যবহারকারী জানিয়েছে। **উপসর্গ:** কোনো restart policy, healthcheck বা alerting নেই। **প্রশ্ন:** (ক) কোন তিনটি ঘাটতি একসাথে এই ঘটনা ঘটিয়েছে? (খ) প্রতিটির সমাধান কী? (গ) একটি ন্যূনতম monitoring ও alerting পরিকল্পনা লিখুন।

অংশ D — ইনসিডেন্ট কেস স্টাডি

১০টি কনটেইনার নিরাপত্তা ও প্রোডাকশন ইনসিডেন্ট কেস স্টাডি

প্রতিটি কেসে ঘটনা, কারণ, প্রভাব, প্রতিক্রিয়া ও শিক্ষা দেওয়া আছে। পড়ার পর নিজেকে জিজ্ঞেস করুন: TechNova-তে একই ঘটনা ঘটলে আমি কী করতাম?

কেস ১ — Docker socket মাউন্ট করে হোস্ট দখল

ঘটনা: TechNova-র একটি ডেভেলপমেন্ট টুল কনটেইনারে `/var/run/docker.sock` মাউন্ট করা হয়েছিল সুবিধার জন্য। একটি দুর্বল প্লাগইন দিয়ে আক্রমণকারী কনটেইনারে ঢুকে daemon-কে নির্দেশ দিয়ে হোস্টে root কনটেইনার চালিয়ে হোস্টের ফাইল পড়ে ফেলে। **কারণ:** socket মাউন্ট = daemon নিয়ন্ত্রণ = কার্যত হোস্ট root। **প্রভাব:** হোস্টের সব secret ও ডেটা ঝুঁকিতে। **প্রতিক্রিয়া:** হোস্ট বিচ্ছিন্ন করা, credential rotate করা, socket মাউন্ট সরানো। **শিক্ষা:** `docker.sock` কখনো অ্যাপ কনটেইনারে মাউন্ট করবেন না; প্রয়োজনে সীমিত API proxy ব্যবহার করুন।

কেস ২ — Image-এ ফাঁস হওয়া secret

ঘটনা: একটি public image-এর layer history-তে `ENV AWS_SECRET=CHANGE-ME-...`-এর মতো আসল secret পাওয়া যায়। আক্রমণকারী image pull করে secret বের করে ক্লাউড অ্যাকাউন্টে ঢোকে। **কারণ:** secret Dockerfile-এ লেখা ও image push করা। **প্রভাব:** ক্লাউড বিল ও ডেটা চুরি। **প্রতিক্রিয়া:** সব credential revoke ও rotate, image মুছে rebuild, secret scanning চালু। **শিক্ষা:** secret কখনো image-এ নয় — শুধু runtime-এ, BuildKit secret mount বা runtime env file-এ।

কেস ৩ — পুরনো base image-এ Log4Shell-জাতীয় দুর্বলতা

ঘটনা: TechNova-র API image ১১ মাস ধরে একই base image ব্যবহার করছিল; একটি critical CVE প্রকাশের পর আক্রমণকারী দুর্বলতা কাজে লাগায়। **কারণ:** base image আপডেট ও scan-এর নিয়ম ছিল না। **প্রভাব:** সার্ভিসে অননুমোদিত প্রবেশ। **প্রতিক্রিয়া:** base image আপডেট, scan CI-তে বাধ্যতামূলক, আক্রান্ত হোস্ট পুনর্নির্মাণ। **শিক্ষা:** নিয়মিত base image আপডেট, CI-তে vulnerability scan ও fail-on-critical নিয়ম।

কেস ৪ — Privileged কনটেইনার থেকে escape

ঘটনা: ডিবাগিংয়ের জন্য `--privileged` দিয়ে চালানো কনটেইনার থেকে আক্রমণকারী হোস্টের ডিভাইসে পৌঁছে হোস্ট নিয়ন্ত্রণ করে। **কারণ:** privileged কনটেইনার হোস্টের প্রায় সব ক্ষমতা পায়। **প্রভাব:** পুরো হোস্ট আপস। **প্রতিক্রিয়া:** হোস্ট পুনর্নির্মাণ, সব credential rotate। **শিক্ষা:** `--privileged` প্রোডাকশনে নিষিদ্ধ; প্রয়োজনে নির্দিষ্ট capability (`--cap-add`) দিন।

কেস ৫ — `latest` tag-এ ভুল deploy ও rollback ব্যর্থতা

ঘটনা: শুক্রবার রাতে `latest` দিয়ে deploy করা হয়; নতুন সংস্করণে বাগ ধরা পড়ে, কিন্তু কোন সংস্করণ আগে চলছিল তা কেউ জানে না, rollback ব্যর্থ হয়। **কারণ:** mutable tag, কোনো digest বা সংস্করণ রেকর্ড নেই। **প্রভাব:** ৬ ঘণ্টা ডাউনটাইম। **প্রতিক্রিয়া:** Git commit থেকে আগের image rebuild করে deploy। **শিক্ষা:** immutable tag/digest ব্যবহার, deploy রেকর্ড রাখা, rollback পরীক্ষা করা।

কেস ৬ — ভলিউম ছাড়া ডেটাবেস: `down -v` বিপর্যয়

ঘটনা: একজন অপারেটর পরীক্ষার জন্য `docker compose down -v` চালিয়ে প্রোডাকশন MongoDB-র volume মুছে ফেলেন। **কারণ:** প্রোডাকশনে volume-এর গুরুত্ব না জানা, ব্যাকআপ ছিল না। **প্রভাব:** ব্যবহারকারীর ডেটা হারিয়ে যায়। **প্রতিক্রিয়া:** ব্যাকআপ থেকে আংশিক পুনরুদ্ধার, গ্রাহকদের জানানো। **শিক্ষা:** প্রোডাকশনে `down -v` নিষিদ্ধ, নিয়মিত ব্যাকআপ ও restore পরীক্ষা, volume-এর নাম ও সুরক্ষা।

কেস ৭ — লগে ডিস্ক পূর্ণ হয়ে পুরো হোস্ট অচল

ঘটনা: একটি অ্যাপ প্রতি সেকেন্ডে হাজার হাজার লগ লিখছিল; JSON-file driver-এ rotation ছিল না, হোস্টের ডিস্ক পূর্ণ হয়ে Docker daemon থেমে যায়। **কারণ:** log rotation কনফিগার করা হয়নি। **প্রভাব:** সব কনটেইনার থেমে যায়। **প্রতিক্রিয়া:** লগ মুছে ডিস্ক মুক্ত করা, rotation চালু। **শিক্ষা:** `max-size` ও `max-file` সেট করুন, লগের পরিমাণ monitor করুন।

কেস ৮ — Root হিসেবে চলা কনটেইনারে আক্রমণ

ঘটনা: API কনটেইনার root হিসেবে চলছিল; একটি file-upload দুর্বলতা দিয়ে আক্রমণকারী কনটেইনারের ভেতরে root ফাইল লিখে persistence তৈরি করে। **কারণ:** `USER` নির্দেশনা ছিল না, filesystem writable ছিল। **প্রভাব:** কনটেইনারে ম্যালওয়্যার স্থায়ী হয়। **প্রতিক্রিয়া:** কনটেইনার পুনর্নির্মাণ, non-root user ও read-only filesystem চালু। **শিক্ষা:** least privilege — non-root user, read-only root filesystem, capability drop।

কেস ৯ — DNS/নেটওয়ার্ক ভুলে মাইক্রোসার্ভিস বিচ্ছিন্ন

ঘটনা: একটি নেটওয়ার্ক পরিবর্তনের পর API আর MongoDB-কে খুঁজে পাচ্ছিল না; সার্ভিস ২ ঘণ্টা বন্ধ ছিল। **কারণ:** কনটেইনারগুলো ভিন্ন network-এ ছিল, hardcode করা IP বদলে গিয়েছিল। **প্রভাব:** পুরো প্ল্যাটফর্ম ডাউন। **প্রতিক্রিয়া:** একই user-defined network-এ ফেরানো, সার্ভিস নাম ব্যবহার। **শিক্ষা:** IP hardcode করবেন না; সার্ভিস নাম, healthcheck ও network ডকুমেন্টেশন রাখুন।

কেস ১০ — Resource limit না থাকায় প্রতিবেশী সার্ভিস ধ্বংস

ঘটনা: একটি রিপোর্ট-জেনারেশন কনটেইনার সব মেমরি খেয়ে ফেলায় MongoDB OOMKilled হয়ে ডেটা লেখা বন্ধ হয়ে যায়। **কারণ:** কোনো কনটেইনারে resource limit ছিল না। **প্রভাব:** ডেটা লেখা ব্যর্থ, ব্যবহারকারী ক্ষতিগ্রস্ত। **প্রতিক্রিয়া:** সব সার্ভিসে memory/CPU limit, MongoDB-র জন্য reservation। **শিক্ষা:** প্রতিটি সার্ভিসে limit ও reservation দিন; `docker stats` দিয়ে monitor করুন।

অংশ E — চূড়ান্ত পরীক্ষা

ব্যবহারিক চূড়ান্ত পরীক্ষা — Docker & Container Engineering

মোট নম্বর: ১০০ • সময়: ৩ ঘণ্টা • পাস নম্বর: ৬০ — তত্ত্ব অংশ নোট বন্ধ রেখে দিন; হাতে-কলমে অংশে নিজের ল্যাব VM ব্যবহার করুন, ইন্টারনেটে ডকুমেন্টেশন দেখা যাবে, কিন্তু অন্যের সমাধান কপি করা যাবে না। সব উদাহরণ TechNova MERN প্ল্যাটফর্মের প্রসঙ্গে।

সেকশন A — তত্ত্ব (৩০ নম্বর, ৪৫ মিনিট)

প্রতিটি প্রশ্নের উত্তর ৪–৬ বাক্যে লিখুন। প্রতিটি প্রশ্ন ৫ নম্বর।

1. VM বনাম কনটেইনার: kernel ভাগ করা, isolation ও startup সময়ের দিক থেকে পার্থক্য ব্যাখ্যা করুন।
2. Namespace (PID, mount, network, user) ও cgroups কীভাবে একসাথে কনটেইনার তৈরি করে — একটি চিত্রসহ ব্যাখ্যা করুন।
3. Docker image layer ও cache কীভাবে কাজ করে? Dockerfile-এর ক্রম কেন build সময় বদলে দেয়?
4. Multi-stage build কেন প্রোডাকশন image-এর জন্য সেরা অভ্যাস? একটি Node.js উদাহরণ দিন।
5. Bridge, host ও none network-এর পার্থক্য এবং প্রতিটির উপযুক্ত ব্যবহার লিখুন।
6. কনটেইনার নিরাপত্তার পাঁচটি স্তর (image, runtime, network, secret, host) সংক্ষেপে ব্যাখ্যা করুন।

সেকশন B — কনফিগারেশন ও ডিজাইন (৩০ নম্বর, ৪৫ মিনিট)

প্রতিটি প্রশ্ন ১০ নম্বর। কোড/ YAML লিখুন ও প্রতিটি গুরুত্বপূর্ণ লাইনের ব্যাখ্যা দিন।

1. TechNova-র Express API-র জন্য একটি multi-stage Dockerfile লিখুন: `node:20-alpine` base, `npm ci` cache-বান্ধব ক্রম, non-root `node` user, `HEALTHCHECK`, exec-form `CMD`, `.dockerignore`-এর তালিকাসহ। কেন প্রতিটি সিদ্ধান্ত নিয়েছেন, তা লিখুন।
2. TechNova MERN স্ট্যাকের (Next.js web, Express api, MongoDB) জন্য একটি সম্পূর্ণ `docker-compose.yml` লিখুন: user-defined network, named volume, healthcheck-ভিত্তিক `depends_on`, resource limit, restart policy, environment (কোনো আসল secret নয়, `CHANGE-ME-...` placeholder), এবং `version:` key ছাড়া। প্রতিটি সার্ভিসের নেটওয়ার্কিং ব্যাখ্যা করুন।
3. একটি প্রোডাকশন deployment পরিকল্পনা লিখুন: immutable tag/digest, registry scan, rolling update, healthcheck, rollback ধাপ, ব্যাকআপ ও monitoring/alerting — কোন ধাপে কী যাচাই করবেন, তা তালিকাসহ দিন।

সেকশন C — হাতে-কলমে কাজ (২৫ নম্বর, ৬০ মিনিট)

নিজের ল্যাব VM-এ করুন ও আউটপুট/স্ক্রিনশট সংরক্ষণ করুন। প্রতিটি কাজ ৫ নম্বর।

1. একটি ছোট Express অ্যাপের multi-stage image build করুন; `docker image ls` ও `docker history` দিয়ে আকার ও লেয়ার দেখান; `node:20` বনাম `node:20-alpine` base-এর আকার তুলনা করুন।

2. `docker compose up -d` দিয়ে api + mongo চালু করুন; `docker ps`, `docker logs` ও `curl` দিয়ে প্রমাণ করুন API MongoDB-র সঙ্গে কথা বলছে; তারপর `docker compose down` (ভলিউম না মুছে) করে আবার চালু করে ডেটা টিকে আছে কি না দেখান।
3. ইচ্ছাকৃতভাবে একটি কনটেইনারে `--memory=128m` সীমা দিয়ে OOM পরীক্ষা করুন; `docker inspect` ও `docker stats` -এর আউটপুট ব্যাখ্যা করুন।
4. একটি image-এ Trivy (বা সমতুল্য scanner) চালিয়ে রিপোর্ট দেখান; একটি high দুর্বলতা পেলে কী করবেন, তা লিখুন।
5. একটি কনটেইনারে `docker exec` দিয়ে ঢুকে `whoami`, `ps` ও নেটওয়ার্ক পরীক্ষা করুন; non-root user ও read-only filesystem যাচাই করুন।

সেকশন D — ইনসিডেন্ট রেসপন্স (১৫ নম্বর, ৩০ মিনিট)

নিচের ঘটনার জন্য একটি লিখিত incident response দিন (১৫ নম্বর): শুক্রবার রাত ১১টায় TechNova-র প্রোডাকশন API কনটেইনার বারবার restart হচ্ছে (`Restarting (1) 5 seconds ago`), MongoDB কনটেইনার চলছে, কিন্তু লগে `MongoServerSelectionError` দেখা যাচ্ছে। লিখুন: (ক) প্রথম ১০ মিনিটে কী করবেন (triage), (খ) নির্ণয়ের ধাপ ও কমান্ড, (গ) সম্ভাব্য তিনটি কারণ ও প্রতিটির সমাধান, (ঘ) ব্যবহারকারীকে কী জানাবেন, (ঙ) ঘটনার পর (post-incident) কী পরিবর্তন করবেন যাতে আবার না ঘটে।

> **পরীক্ষার নিয়ম:** কোনো আসল secret, token বা পাসওয়ার্ড পরীক্ষার উত্তরে লিখবেন না — সব জায়গায় `CHANGE-ME-...` placeholder ব্যবহার করুন। হাতে-কলমে অংশে প্রোডাকশন সার্ভার ব্যবহার করবেন না, শুধু ল্যাব VM ব্যবহার করুন।

সেকশন	নম্বর	সময়
A — তত্ত্ব	৩০	৪৫ মিনিট
B — কনফিগারেশন ও ডিজাইন	৩০	৪৫ মিনিট
C — হাতে-কলমে	২৫	৬০ মিনিট
D — ইনসিডেন্ট রেসপন্স	১৫	৩০ মিনিট
মোট	**১০০**	**৩ ঘণ্টা**

অংশ F — উত্তর নির্দেশিকা (১ম অর্ধ)

উত্তর নির্দেশিকা — MCQ ১–৬০ ও সংক্ষিপ্ত প্রশ্ন ১–১৫

প্রতিটি উত্তরে সঠিক বিকল্পের পাশাপাশি **কেন** সেটি সঠিক এবং অন্য বিকল্পগুলো কেন ভুল, তা ব্যাখ্যা করা হয়েছে। অক্ষর মুখস্থ না করে ব্যাখ্যাটি বুঝুন।

MCQ ১–২০: VM, namespace, cgroups — উত্তর ও ব্যাখ্যা

1. **ক** — VM-এ hypervisor প্রতিটি VM-এর জন্য আলাদা kernel ও OS চালায়; কনটেইনার host kernel ভাগ করে, শুধু namespace/cgroups দিয়ে আলাদা থাকে। তাই কনটেইনার হালকা, VM ভারী।
2. **খ** — কনটেইনার পূর্ণ OS boot করে না, শুধু প্রসেস শুরু করে; তাই একই হোস্টে অনেক কনটেইনার চলে। RAM/CPU তবুও লাগে — "লাগে না" বিকল্পগুলো ভুল।
3. **খ** — কনটেইনার শুধু অ্যাপ প্রসেস শুরু করে, OS boot-এর সময় লাগে না; VM-এ kernel ও OS boot করতে হয়।
4. **খ** — PID namespace প্রতিটি কনটেইনারকে নিজের PID 1-সহ আলাদা প্রসেস-তালিকা দেয়; হোস্টের প্রসেস দেখা যায় না।
5. **ক** — Mount namespace ছাড়া কনটেইনার নিজের root filesystem দৃষ্টিভঙ্গি পেত না; সবাই একই `/` দেখত।
6. **খ** — Network namespace ইন্টারফেস, IP, রাউটিং ও পোর্ট আলাদা করে; তাই দুটি কনটেইনার একই পোর্ট ব্যবহার করতে পারে।
7. **খ** — UTS namespace হোস্টনেম/ডোমেইন নাম আলাদা করে; `hostname` কমান্ডের ফল কনটেইনার-নির্দিষ্ট হয়।
8. **ক** — User namespace কনটেইনারের UID 0-কে হোস্টে অ-root UID-তে ম্যাপ করতে পারে; escape হলেও হোস্টে root পাওয়া যায় না।
9. **খ** — cgroups সম্পদ সীমিত ও হিসাব করে; namespace আলাদা করে, cgroups সীমা দেয় — দুটি ভিন্ন কাজ।
10. **গ** — মেমরি সীমা ছাড়ালে kernel OOM killer প্রসেস মেরে ফেলে; কনটেইনার `Exited (137)` হয়।
11. **খ** — `--cpus=1.5` মানে মোট দেড় কোর-সমপরিমাণ CPU সময়ের ঊর্ধ্বসীমা, একচেটিয়া কোর বরাদ্দ নয়।
12. **খ** — CPU সমস্যায় CPU সীমা দিতে হয়; `--memory` মেমরির জন্য, `--read-only` ফাইলের জন্য।
13. **খ** — Kernel ভাগ করা মানেই kernel দুর্বলতা সব কনটেইনারকে প্রভাবিত করতে পারে; VM-এ এই ঝুঁকি কম।
14. **খ** — অবিশ্বস্ত কোড বা আলাদা kernel দরকার হলে VM-এর strong isolation যুক্তিযুক্ত; সব কাজে কনটেইনার নয়।
15. **ক** — `unshare` নতুন namespace তৈরি করে; এটি namespace ধারণা বোঝার সরাসরি উপায়।
16. **খ** — হোস্টের প্রসেস দেখা গেলে PID namespace ভাগ করা হয়েছে (`--pid=host`); স্বাভাবিক কনটেইনারে এমন হয় না।
17. **খ** — cgroups v2-এ `memory.max` ও `cpu.max` কঠোর ঊর্ধ্বসীমা; শুধু তথ্য দেখায় না।
18. **খ** — মেমরি সীমা + অ্যাপের cache সেটিং পর্যালোচনা করাই টেকসই সমাধান; কনটেইনার মুছে ফেলা সমস্যা লুকায়।
19. **ক** — IPC namespace shared memory/semaphore আলাদা করে; না থাকলে কনটেইনারগুলো একে অপরের shared memory দেখতে পেত।
20. **খ** — কনটেইনার VM নয়; নিজের kernel নেই, এটি isolation-সহ প্রসেস-সমষ্টি — "হালকা VM" বলা ধারণাগত ভুল।

MCQ ২১–৪০: Docker আর্কিটেকচার, ইমেজ, লেয়ার — উত্তর ও ব্যাখ্যা

1. **খ** — `dockerd` API শুনে কনটেইনার, ইমেজ, নেটওয়ার্ক ও ভলিউম পরিচালনা করে এবং containerd-কে নির্দেশ দেয়।
2. **খ** — Client daemon-এর সঙ্গে REST API-তে কথা বলে, সাধারণত Unix socket বা TCP-তে; ইমেইল/SSH দিয়ে নয়।
3. **খ** — `docker.sock` মাউন্ট করলে কনটেইনার daemon নিয়ন্ত্রণ করতে পারে — নতুন privileged কনটেইনার চালিয়ে হোস্ট দখল করা যায়।
4. **খ** — containerd কনটেইনার lifecycle পরিচালনা করে; Docker daemon তার উপরের স্তর।
5. **খ** — runc OCI runtime; namespace ও cgroups প্রয়োগ করে আসল প্রসেস শুরু করে।
6. **খ** — OCI ইমেজ ও runtime-এর উন্মুক্ত মান দেয়; ফলে Docker, containerd, Podman পরস্পরের সঙ্গে কাজ করে।
7. **খ** — Image read-only লেয়ারের সংগ্রহ; কনটেইনার হলো সেই image থেকে তৈরি চলমান instance।
8. **ক** — প্রতিটি Dockerfile নির্দেশনা (কিছু ব্যতিক্রম ছাড়া) একটি নতুন লেয়ার যোগ করে।
9. **ক** — লেয়ার content-addressable; একই ইনপুটে একই hash, তাই cache পুনঃব্যবহার হয়।
10. **খ** — উপরের লেয়ার নিচের লেয়ারের উপর নির্ভর করে; নিচের hash বদলালে উপরের cache অকার্যকর হয়।
11. **খ** — একাধিক tag একই লেয়ার-সমষ্টিকে নির্দেশ করে; ডিস্কে একটিই কপি থাকে।
12. **খ** — Writable layer image লেয়ারগুলোর উপরে থাকে; কনটেইনার-নির্দিষ্ট ও পাতলা।
13. **খ** — কনটেইনার মুছলে writable layer-এর ডেটা যায়; স্থায়ী ডেটা volume-এ রাখতে হয়।
14. **খ** — `docker commit` হাতে বদলানো অবস্থা ধরে রাখে; Dockerfile ছাড়া তা পুনরাবৃত্তিযোগ্য নয়।
15. **খ** — Alpine ছোট, কম প্যাকেজ, কম CVE, দ্রুত pull; তবে musl/glibc সামঞ্জস্য যাচাই করতে হয়।
16. **খ** — Alpine musl-ভিত্তিক ও ছোট; পূর্ণ `node:20` Debian-ভিত্তিক, বড় কিন্তু glibc সামঞ্জস্য ভালো।
17. **খ** — Digest content-এর hash; tag বদলে গেলেও একই image আসে।
18. **খ** — `prune -a` ব্যবহৃত হচ্ছে না এমন সব image মুছে ফেলে; চলমান কনটেইনারের image মুছে না।
19. **খ** — ১.২ GB image-এর প্রথম সন্দেহ বড় base image, build টুল চূড়ান্ত image-এ থাকা, বা অপ্রয়োজনীয় ফাইল কপি হওয়া।
20. **ক** — overlay2 একাধিক read-only লেয়ারকে একটিমাত্র filesystem দৃষ্টিভঙ্গিতে জোড়া দেয়।

MCQ ৪১–৬০: Dockerfile, cache, multi-stage — উত্তর ও ব্যাখ্যা

1. **খ** — কম বদলানো ধাপ আগে রাখলে cache বেশি কাজ করে; সোর্স কোড বদলালেও dependency লেয়ার আবার তৈরি হয় না।
2. **খ** — `package*.json` আগে কপি করলে সোর্স বদলালেও `npm ci` লেয়ার cache থেকে আসে।
3. **খ** — `npm ci` lockfile অনুযায়ী ঠিক সংস্করণ ইনস্টল করে; `npm install` lockfile আপডেট করতে পারে।
4. **খ** — `.dockerignore` build context ছোট করে; `node_modules` ও `.git` বাদ দিলে build দ্রুত হয়।
5. **খ** — বড় context প্রতিবার daemon-এ পাঠাতে হয়; সময় ও ব্যান্ডউইথ খরচ হয়।
6. **খ** — `WORKDIR` না দিলে পরের নির্দেশনা `/`-তে চলে; পথ-ভুল সহজ হয়।
7. **খ** — `ENTRYPOINT` মূল কমান্ড, `CMD` ডিফল্ট আর্গুমেন্ট; `docker run`-এর আর্গুমেন্ট `CMD`-কে বদলে দেয়।
8. **খ** — Exec form-এ Node সরাসরি PID 1 হয় ও SIGTERM পায়; shell form-এ shell PID 1 হয়, signal পৌঁছায় না।

9. **খ** — Signal না ধরলে Docker ১০ সেকেন্ড অপেক্ষা করে SIGKILL দেয়; এটাই ঝুলে থাকার কারণ।
10. **খ** — `ENV` image-এ স্থায়ী হয় ও `docker inspect`-এ দেখা যায়; secret কখনো `ENV`-এ নয়।
11. **খ** — `ARG` শুধু build সময়ে, `ENV` runtime-এও থাকে — এই পার্থক্যই secret-ঝুঁকি নির্ধারণ করে।
12. **খ** — Multi-stage-এ build টুল চূড়ান্ত image-এ যায় না; ছোট ও নিরাপদ runtime image হয়।
13. **খ** — `COPY --from=builder` শুধু প্রয়োজনীয় artifact নেয়, পুরো builder image নয়।
14. **খ** — BuildKit cache mount dependency cache পুনঃব্যবহার করে, কিন্তু cache image-এ থাকে না।
15. **খ** — এক লাইনে ইনস্টল ও cache পরিষ্কার করলে apt cache আলাদা লেয়ারে থেকে যায় না।
16. **ক** — আলাদা লেয়ারে `apt-get update` cache হলে পুরনো package list ব্যবহার হতে পারে।
17. **খ** — `USER node` কনটেইনারকে root-এর বদলে অ-root হিসেবে চালায়; escape-এর ক্ষতি কমে।
18. **খ** — `HEALTHCHECK` নিয়মিত কমান্ড চালিয়ে কনটেইনার সত্যিই সাড়া দিচ্ছে কি না যাচাই করে।
19. **খ** — `EXPOSE` শুধু metadata; পোর্ট প্রকাশ করতে `-p` বা Compose `ports:` লাগে।
20. **খ** — প্রথমেই `COPY . .` থাকলে যেকোনো ফাইল বদলালে সব লেয়ার আবার তৈরি হয়।

সংক্ষিপ্ত প্রশ্ন ১–১৫ — উত্তর নির্দেশিকা

1. **VM বনাম কনটেইনার:** (১) VM-এ আলাদা kernel, কনটেইনারে host kernel ভাগ; (২) VM boot-এ মিনিট লাগে, কনটেইনার সেকেন্ডে শুরু হয়; (৩) VM-এ strong isolation, কনটেইনারে হালকা কিন্তু kernel-নির্ভর isolation। TechNova-তে দ্রুত deploy-এর জন্য কনটেইনার, অবিশ্বস্ত কোডের জন্য VM।
2. **Namespace:** PID namespace প্রসেস-তালিকা আলাদা করে, mount namespace filesystem দৃষ্টিভঙ্গি আলাদা করে। একসাথে তারা কনটেইনারকে মনে করায় সে একাই মেশিনে আছে, যদিও হোস্টে আরও প্রসেস চলছে।
3. **cgroups:** CPU, মেমরি, I/O-র ব্যবহার সীমিত ও হিসাব করে। সীমা না দিলে একটি কনটেইনার সব সম্পদ খেয়ে প্রতিবেশী সার্ভিসকে ধীর করে দিতে পারে; সীমা দিলে “noisy neighbour” সমস্যা কমে।
4. **Daemon/containerd/runc:** Docker daemon API ও উচ্চ-স্তরের পরিচালনা করে, containerd lifecycle পরিচালনা করে, runc OCI অনুযায়ী আসল প্রসেস শুরু করে। তিন স্তর আলাদা থাকায় Docker-এর নিচের runtime বদলানো যায়।
5. **Image বনাম writable layer:** Image লেয়ার read-only ও ভাগ করা যায়; writable layer কনটেইনার-নির্দিষ্ট, image লেয়ারের উপরে। কনটেইনার মুছলে writable layer যায়, image থাকে।
6. **Layer cache:** প্রতিটি লেয়ারের hash তার নির্দেশনা ও ইনপুটের উপর নির্ভর করে। কোনো লেয়ার বদলালে তার উপরের সব লেয়ারের cache ভাঙে; তাই স্থিতিশীল ধাপ আগে রাখতে হয়।
7. **Dockerfile ক্রম:** কম বদলানো ধাপ (base image, dependency install) আগে, ঘন ঘন বদলানো ধাপ (সোর্স কপি) পরে রাখলে cache বেশি কাজ করে ও build দ্রুত হয়।
8. **.dockerignore:** Build context থেকে `node_modules`, `.git`, `.env`, লগ ফাইল বাদ দেয়। ফলে context ছোট হয়, build দ্রুত হয়, এবং ভুল করে secret বা বড় ফাইল image-এ যাওয়া কমে।
9. **Multi-stage (MERN):** প্রথম stage-এ TypeScript/Next.js build হয়, দ্বিতীয় stage-এ শুধু build ফলাফল ও runtime dependency নেওয়া হয়। ফলে চূড়ান্ত image-এ compiler, dev dependency ও সোর্স থাকে না — ছোট ও নিরাপদ।
10. **CMD বনাম ENTRYPOINT:** `ENTRYPOINT` কনটেইনারের মূল কমান্ড, `CMD` তার ডিফল্ট আর্গুমেন্ট। `docker run technova/api --port 4000` দিলে `CMD` বদলে যায়, `ENTRYPOINT` থাকে।

11. **Exec বনাম shell form:** Exec form-এ অ্যাপ সরাসরি PID 1 হয় ও SIGTERM পায়, graceful shutdown হয়। Shell form-এ shell PID 1 হয়, signal অ্যাপে পৌঁছায় না, `docker stop` ঝুলে SIGKILL-এ থামে।
12. **Tag বনাম digest:** Tag মানুষের পড়ার জন্য সহজ কিন্তু mutable; digest অপরিবর্তনীয় কিন্তু পড়া কঠিন। প্রোডাকশনে tag + digest দুটোই রেকর্ড করা সেরা অভ্যাস।
13. **User-defined bridge:** এতে embedded DNS থাকে, কনটেইনার নামে একে অপরকে খুঁজে পায়; default bridge-এ তা হয় না। এছাড়া নেটওয়ার্ক আলাদা করে সার্ভিসগুলোকে বিচ্ছিন্ন করা যায়।
14. **DNS resolution:** একই user-defined network-এর কনটেইনারগুলো embedded DNS (127.0.0.11)-এর মাধ্যমে নাম resolve করে। তাই API `mongodb://mongo:27017` দিয়ে MongoDB-কে ডাকতে পারে, IP hardcode করতে হয় না।
15. **-p 8080:3000:** হোস্টের ৮০৮০ পোর্ট কনটেইনারের ৩০০০ পোর্টে যুক্ত হয়। বাইরে থেকে `http://host:8080` ডাকলে কনটেইনারের ৩০০০ পোর্টে পৌঁছায়।

অংশ F — উত্তর নির্দেশিকা (২য় অর্ধ)

উত্তর নির্দেশিকা — MCQ ৬১–১২০, সংক্ষিপ্ত ১৬–৩০, পরিস্থিতি, কেস স্টাডি ও চূড়ান্ত পরীক্ষা

MCQ ৬১–৮০: Registry, নেটওয়ার্কিং, DNS — উত্তর ও ব্যাখ্যা

1. **খ** — Registry image সংরক্ষণ, tag/digest দিয়ে সংস্করণ চিহ্নিতকরণ ও বিতরণ করে; কনটেইনার চালায় না।
2. **খ** — নিজস্ব কোডের image বাইরে প্রকাশ করতে না চাইলে private registry দরকার; অ্যাক্সেস নিয়ন্ত্রণ ও audit থাকে।
3. **খ** — Push-এর আগে image-টি registry-র নামে tag করা ও `docker login` দিয়ে authenticate করা দরকার।
4. **খ** — `latest` mutable; কোন সংস্করণ চলছে তা নিশ্চিত করা যায় না, rollback অনিশ্চিত হয়।
5. **খ** — Digest content-এর hash; একই digest সবসময় একই image দেয়।
6. **খ** — Trivy OS প্যাকেজ ও লাইব্রেরিতে জানা CVE খোঁজে; কোডের স্টাইল বা নেটওয়ার্ক খোঁজে না।
7. **খ** — একই user-defined bridge network-এ কনটেইনার নাম দিয়ে DNS resolution হয়; IP মুখস্থ করতে হয় না।
8. **খ** — Default bridge legacy; এতে embedded DNS নেই, তাই নামে resolution হয় না।
9. **ক** — `-p 8080:3000` মানে হোস্ট ৮০৮০ → কনটেইনার ৩০০০; ক্রমটি হোস্ট:কনটেইনার।
10. **খ** — Host network-এ কনটেইনার হোস্টের namespace ভাগ করে; isolation কমে, পোর্ট সংঘাত হয়।
11. **খ** — None network কনটেইনারকে সম্পূর্ণ বিচ্ছিন্ন রাখে; একবারের batch কাজে উপযুক্ত।
12. **খ** — Overlay network একাধিক হোস্টের কনটেইনারকে একই virtual network-এ যুক্ত করে।
13. **খ** — নাম resolve না হলে প্রথমে যাচাই করুন দুটি কনটেইনার একই user-defined network-এ আছে কি না।
14. **খ** — Embedded DNS (127.0.0.11) কনটেইনারের নাম-অনুরোধ ধরে একই network-এর কনটেইনার resolve করে।
15. **ক** — `--dns` কনটেইনারের DNS সার্ভার বদলে দেয়; হোস্টের DNS বদলায় না।
16. **খ** — একটি কনটেইনার ইন্টারনেটে যেতে না পারলে network, DNS বা egress নিয়ম যাচাই করুন; image/CPU নয়।
17. **খ** — `network inspect`-এ subnet, gateway, সংযুক্ত কনটেইনার ও IP দেখা যায়।
18. **খ** — পোর্ট ইতিমধ্যে ব্যবহৃত হলে bind ব্যর্থ হয়; অন্য হোস্ট পোর্ট বেছে নিন।
19. **খ** — Service discovery হলো সার্ভিসগুলো কীভাবে একে অপরকে খুঁজে পায় — নাম, DNS বা registry-র মাধ্যমে।
20. **খ** — Ingress/reverse proxy বাইরের ট্রাফিক এক প্রবেশপথ দিয়ে নিয়ে ভেতরের সার্ভিসে রাউট করে; TLS ও রাউটিং এক জায়গায় থাকে।

MCQ ৮১–১০০: ভলিউম, Compose, MERN, secrets — উত্তর ও ব্যাখ্যা

1. **খ** — Named volume Docker পরিচালনা করে, হোস্ট পথ থেকে স্বাধীন; bind mount হোস্টের নির্দিষ্ট পথের সঙ্গে বাঁধা।
2. **খ** — কনটেইনারের ভেতরের ডেটা কনটেইনার মুছলে যায়; MongoDB-র ডেটা volume-এ রাখতে হয়।
3. **খ** — `-v mongo-data:/data/db` মানে named volume `mongo-data` কনটেইনারের `/data/db`-তে মাউন্ট।
4. **খ** — Bind mount-এ হোস্টের সোর্স কোড কনটেইনারে মাউন্ট করে hot reload-এ কাজ করা যায়; প্রোডাকশন ডেটার জন্য নয়।

5. **ক** — UID/GID হোস্ট ও কনটেইনারে ভিন্ন হতে পারে; কনটেইনারের root লেখা ফাইল হোস্টে root-মালিকানাধীন হয়।
6. **খ** — Read-only volume কনটেইনারকে ডেটা বদলাতে দেয় না; অখণ্ডতা ও নিরাপত্তা রক্ষা করে।
7. **খ** — `volume prune` ব্যবহৃত-না-হওয়া volume স্থায়ীভাবে মুছে ফেলে; আগে ডেটা যাচাই করুন।
8. **খ** — Compose-এ `services:` প্রতিটি কনটেইনার (api, web, mongo) ঘোষণা করে।
9. **খ** — `depends_on` শুধু শুরু করার ক্রম দেয়; সার্ভিস প্রস্তুত হয়েছে কি না, তা নিশ্চিত করে না।
10. **খ** — Healthcheck-সহ `condition: service_healthy` দিলে নির্ভরশীল সার্ভিস সত্যিই সাড়া দেওয়ার পরেই পরেরটি শুরু হয়।
11. **খ** — `environment:` YAML-এ সরাসরি লেখা হয়, `env_file:` আলাদা ফাইল থেকে পড়ে; ফাইলটি Git-এ কমিট করা উচিত নয়।
12. **খ** — API MongoDB প্রস্তুত হওয়ার আগেই শুরু হলে সংযোগ ব্যর্থ হয়; healthcheck-ভিত্তিক `depends_on` বা retry দরকার।
13. **খ** — একই user-defined network-এ সার্ভিস নাম দিয়ে যোগাযোগ করাই সেরা উপায়; host network বা IP hardcode নয়।
14. **খ** — `NEXT_PUBLIC_*` build সময়ে bundle-এ ঢুকে যায়; runtime secret কখনো build-time env-এ রাখা উচিত নয়।
15. **খ** — Secret image layer-এ থাকলে image যার হাতে যাবে, সে-ই তা দেখতে পাবে; history-তেও থেকে যায়।
16. **খ** — Runtime secret শুধু runtime-এ দেওয়া হয়, image-এ থাকে না — এটাই নিরাপদ পদ্ধতি।
17. **খ** — `-d` detached mode; সার্ভিস পেছনে চলে, টার্মিনাল মুক্ত থাকে।
18. **খ** — `down -v` কনটেইনার, নেটওয়ার্ক ও named volume মুছে ফেলে; ডেটা হারানোর ঝুঁকি আছে।
19. **খ** — `restart: unless-stopped` ব্যর্থ হলে বা হোস্ট রিস্টার্ট হলে সার্ভিস আবার চালু করে; প্রোডাকশন-ঘেঁষা সার্ভিসে উপযুক্ত।
20. **খ** — আধুনিক Compose v2-এ `version:` উপেক্ষা করা হয় ও সতর্কবার্তা আসতে পারে; সরিয়ে ফেলুন।

MCQ ১০১–১২০: নিরাপত্তা, resource, লগিং, প্রোডাকশন, ক্লাউড — উত্তর ও ব্যাখ্যা

1. **খ** — Root হিসেবে চলা কনটেইনার থেকে escape হলে হোস্টে root পাওয়ার সম্ভাবনা বাড়ে; non-root user এই ঝুঁকি কমায়।
2. **খ** — `--read-only` root filesystem লেখা বন্ধ করে; আক্রমণকারী ফাইল বদলাতে পারে না।
3. **ক** — `--cap-drop=ALL` সব capability বাদ দিয়ে শুধু প্রয়োজনীয়টি যোগ করে — least privilege নীতি।
4. **খ** — `--privileged` হোস্টের প্রায় সব ক্ষমতা দেয়; প্রোডাকশনে কার্যত হোস্ট root দেওয়ার সমান।
5. **খ** — প্রতিটি প্যাকেজ সম্ভাব্য CVE ও আক্রমণ-ক্ষেত্র বাড়ায়; ছোট image নিরাপদ।
6. **খ** — Critical/high দুর্বলতা পেলে build আটকে দিলে দুর্বল image প্রোডাকশনে যায় না।
7. **খ** — Exit 137 মানে SIGKILL, সাধারণত OOM; মেমরি সীমা বা হোস্ট মেমরি যাচাই করুন।
8. **খ** — `docker stats`-এ চলমান কনটেইনারের CPU, মেমরি, নেটওয়ার্ক ও block I/O live দেখা যায়।
9. **খ** — অ্যাপ stdout/stderr-এ লগ লিখলে Docker তা সংগ্রহ করে; ফাইলে লিখলে Docker তা দেখে না।
10. **খ** — Rotation না করলে JSON লগ ফাইল ডিস্ক ভরে ফেলতে পারে; `max-size` / `max-file` সেট করুন।
11. **খ** — Healthcheck ব্যর্থ হলে orchestrator কনটেইনারকে unhealthy ধরে রিস্টার্ট বা ট্রাফিক থেকে সরাতে পারে।
12. **খ** — Rolling update-এ একবারে কয়েকটি কনটেইনার বদলানো হয়; সমস্যা হলে থামানো যায়।

13. **খ** — Blue-green-এ পুরনো পরিবেশ চালু থাকে; ট্রাফিক ফিরিয়ে দিলেই rollback হয়।
14. **খ** — Canary-তে নতুন সংস্করণ প্রথমে অল্প ট্রাফিকে পরীক্ষা করে মেট্রিক দেখে ধীরে ধীরে বাড়ানো হয়।
15. **খ** — Test পাসের পরে image build ও push হয়, deploy-এর আগে; একই artifact সব environment-এ ব্যবহৃত হয়।
16. **খ** — Kubernetes-এর Pod, image, volume, network ও healthcheck ধারণা Docker-এর ভিত্তির উপর দাঁড়িয়ে আছে।
17. **ক** — ECS, ACI, Cloud Run-এ সার্ভার পরিচালনা ছাড়াই কনটেইনার চালানো যায়; তবে খরচ ও সীমা যাচাই করতে হয়।
18. **খ** — `latest` দিয়ে deploy করলে কোন সংস্করণ চলছিল তা জানা যায় না; নির্দিষ্ট সংস্করণে ফেরা কঠিন হয়।
19. **খ** — রাতে থেমে গিয়ে কেউ না জানলে healthcheck, restart policy, monitoring ও alerting-এর অভাবই মূল ঘাটতি।
20. **খ** — ক্যাপস্টোনের প্রমাণ হলো multi-stage build, healthcheck, resource limit, non-root user, volume-এ ডেটা, Compose-এ পুরো স্ট্যাক ও rollback পরিকল্পনা — সব একসাথে কাজ করা।

সংক্ষিপ্ত প্রশ্ন ১৬–৩০ — উত্তর নির্দেশিকা

1. **Volume বনাম bind mount বনাম tmpfs:** Named volume Docker পরিচালনা করে ও ডেটা স্থায়ী রাখে (MongoDB); bind mount হোস্টের পথের সঙ্গে বাঁধা, dev-এ সোর্স শেয়ারে উপযুক্ত; tmpfs মেমরিতে থাকে, রিস্টার্টে মুছে যায়, সংবেদনশীল অস্থায়ী ডেটার জন্য।
2. **depends_on-এর সীমা:** এটি শুধু শুরু করার ক্রম দেয়, readiness দেয় না। সমাধান: healthcheck-সহ `condition: service_healthy`, অথবা অ্যাপে retry/backoff যুক্ত করা।
3. **Healthcheck:** `test: ["CMD", "node", "healthcheck.js"]` বা `curl -f http://localhost:3000/health` নিয়মিত চালিয়ে API সাড়া দিচ্ছে কি না যাচাই করে; ব্যর্থ হলে কনটেইনার unhealthy হয়।
4. **Env বনাম secret:** সাধারণ কনফিগারেশন (যেমন `NODE_ENV`) env variable-এ; পাসওয়ার্ড, token, connection string secret হিসেবে runtime-এ দিন, কখনো image বা Git-এ নয়।
5. **Next.js env:** `NEXT_PUBLIC_*` build সময়ে client bundle-এ ঢুকে যায় ও সবার কাছে দৃশ্যমান হয়; সার্ভার-শুধু secret runtime env-এ রাখতে হয়।
6. **Non-root user:** কনটেইনার root হিসেবে চললে escape-এর ক্ষতি বেশি হয়। `USER node` দিলে আক্রমণকারী root ক্ষমতা পায় না; ফাইল permission-ও সীমিত থাকে।
7. **Read-only ও cap-drop:** `--read-only` filesystem লেখা বন্ধ করে, `--cap-drop=ALL` অপ্রয়োজনীয় kernel ক্ষমতা বাদ দেয়। দুটো একসাথে আক্রমণের সুযোগ অনেক কমায়।
8. **Image scanning:** Trivy CI-তে চালালে দুর্বল image প্রোডাকশনে যাওয়ার আগেই ধরা পড়ে। Critical/high পেলে build fail করানো উচিত, নইলে scan শুধু রিপোর্ট হয়ে থাকে।
9. **OOMKilled বনাম exit 1:** 137 মানে SIGKILL, সাধারণত OOM; `docker inspect`-এ `OOMKilled: true` দেখা যায়। Exit 1 মানে অ্যাপ নিজে ত্রুটি নিয়ে থেমেছে; লগে অ্যাপের ত্রুটি দেখতে হয়।
10. **stdout/stderr লগ:** Docker কনটেইনারের stdout/stderr সংগ্রহ করে; ফাইলে লেখা লগ Docker দেখে না। তাই অ্যাপ stdout-এ structured (JSON) লগ লিখলে সংগ্রহ, অনুসন্ধান ও central logging সহজ হয়।
11. **Log rotation:** JSON-file driver-এ rotation না করলে লগ ফাইল বাড়তে বাড়তে ডিস্ক পূর্ণ করে Docker daemon-ই থেমে যেতে পারে। `max-size: 10m` ও `max-file: 3` সেট করুন।
12. **Rolling/blue-green/canary:** Rolling ধীরে ধীরে বদলায়; blue-green দুটি পরিবেশ রেখে একবারে ট্রাফিক বদলায়, rollback সহজ; canary অল্প ট্রাফিকে পরীক্ষা করে ধীরে ধীরে বাড়ায়।

13. **একই artifact:** প্রতিটি environment-এ নতুন করে build করলে সংস্করণ ভিন্ন হতে পারে; একই image সব জায়গায় ব্যবহার করলে “আমার মেশিনে চলছিল” সমস্যা কমে ও rollback নিশ্চিত হয়।
14. **Healthcheck + restart + monitoring:** Healthcheck সমস্যা ধরে, restart policy স্বয়ংক্রিয়ভাবে ফিরিয়ে আনে, monitoring/alerting মানুষকে জানায়। একটিও না থাকলে ব্যর্থতা অজানা থেকে যায়।
15. **MERN তিন কনটেইনারে ভাগ:** সুবিধা — প্রতিটি সার্ভিস আলাদাভাবে আপডেট, স্কেল ও রিস্টার্ট করা যায়। ঝুঁকি — নেটওয়ার্কিং, startup ক্রম ও ডেটা স্থায়িত্বের জটিলতা বাড়ে; Compose ও healthcheck দিয়ে সামলাতে হয়।

পরিস্থিতি ১–২০ — সংক্ষিপ্ত সমাধান নির্দেশিকা

1. **১:** কারণ `express` ইনস্টল হয়নি — সম্ভবত `npm ci` ব্যর্থ হয়েছে বা `node_modules` কপি হয়নি। যাচাই: `docker logs api`, Dockerfile-এ `npm ci` ধাপ। সমাধান: dependency লেয়ার ঠিক করা, `.dockerignore` যাচাই।
2. **২:** দখলকারী খুঁজতে `ss -tlnp | grep 3000` বা `lsof -i :3000`। সমাধান: অন্য হোস্ট পোর্ট (`-p 3001:3000`) বা দখলকারী প্রসেস থামানো। প্রোডাকশনে পোর্ট বরাদ্দ নথিভুক্ত রাখুন।
3. **৩:** কনটেইনারের `localhost` মানে নিজের কনটেইনার, MongoDB নয়। সঠিক string: `mongodb://mongo:27017/technova` (একই network-এ)। প্রস্তুতির জন্য healthcheck-ভিত্তিক `depends_on` বা retry।
4. **৪:** 137 = SIGKILL, OOM নির্দেশ করে। `docker stats` ও `docker inspect`-এ `OOMKilled` দেখুন। সীমা বাড়ানোর আগে memory leak বা অতিরিক্ত cache পর্যালোচনা করুন।
5. **৫:** Default bridge-এ embedded DNS নেই। সমাধান: user-defined network তৈরি করে দুটি কনটেইনার তাতে যুক্ত করুন; Compose-এ ডিফল্টভাবেই তা হয়।
6. **৬:** প্রথমেই `COPY . .` থাকায় যেকোনো পরিবর্তনে cache ভাঙে। সমাধান: `package*.json` আগে কপি করে `npm ci` করুন, তারপর সোর্স কপি করুন; `.dockerignore` যোগ করুন।
7. **৭:** কারণ: বড় base image, build টুল চূড়ান্ত image-এ, dev dependency। সমাধান: multi-stage, `alpine` base, `npm ci --omit=dev`। যাচাই: `docker history technova/api`।
8. **৮:** `down -v` named volume মুছে ফেলেছে; ব্যাকআপ ছাড়া ডেটা ফেরানো যায় না। ভবিষ্যতে: প্রোডাকশনে `down -v` নিষিদ্ধ, নিয়মিত ব্যাকআপ ও restore পরীক্ষা।
9. **৯:** Shell form `CMD`-এ shell PID 1 হয়, SIGTERM অ্যাপে পৌঁছায় না। সমাধান: exec form `CMD ["node", "server.js"]` ও Node-এ SIGTERM handler।
10. **১০:** কারণ: login হয়নি বা token-এর মেয়াদ শেষ, অথবা repository-র নাম ভুল। যাচাই: `docker login` আবার করা, tag যাচাই। CI-তে registry credential secret হিসেবে দিন।
11. **১১:** শুধু লাইন মুছলে পুরনো layer history-তে secret থেকে যায়। করতে হবে: secret rotate/revoke, image মুছে rebuild, secret scanning চালু। ভবিষ্যতে: BuildKit secret mount বা runtime env।
12. **১২:** কারণ: JSON-file log-এ rotation নেই। তাৎক্ষণিক: পুরনো লগ মুছে ডিস্ক মুক্ত করা। স্থায়ী: `daemon.json`-এ `max-size` / `max-file` সেট করা।
13. **১৩:** Healthcheck কমান্ডে `curl` নেই বলে ব্যর্থ হচ্ছে, অ্যাপ ঠিক আছে। সমাধান: `curl` ইনস্টল করা বা Node-ভিত্তিক healthcheck লেখা। Endpoint হালকা ও দ্রুত হওয়া উচিত।
14. **১৪:** Limit না থাকায় ব্যাচ কনটেইনার সব CPU খেয়েছে। সমাধান: API-তে `--cpus` ও ব্যাচে সীমা দিন। `--cpus` কঠোর সীমা, `--cpu-shares` আপেক্ষিক ওজন।

15. **১৫:** ভুল `.env` ফাইল ব্যবহার হয়েছে। সমাধান: প্রতি environment-এ আলাদা env ফাইল, নামে environment লেখা, deploy-এর আগে connection string যাচাই। প্রভাব সীমিত করতে ডেটাবেস ব্যবহারকারীকে environment-ভিত্তিক সীমিত করুন।
16. **১৬:** কারণ: ধীর নেটওয়ার্ক, বড় image, registry সমস্যা। নির্ণয়: `docker pull`-এর ত্রুটি, নেটওয়ার্ক পরীক্ষা। উপায়: ছোট image, registry mirror, layer cache।
17. **১৭:** হোস্ট ডিরেক্টরির মালিক root, কনটেইনার non-root — permission মেলে না। সমাধান: মালিকানা ঠিক করা বা named volume ব্যবহার করা; named volume-এ Docker নিজে permission সামলায়।
18. **১৮:** `latest` mutable বলে কোন সংস্করণ চলছিল তা জানা যায় না। তাৎক্ষণিক: Git commit থেকে আগের image rebuild করে deploy। ভবিষ্যতে: immutable tag/digest ও deploy রেকর্ড।
19. **১৯:** `-v /:/host` + `--privileged` মানে হোস্টের পুরো filesystem ও ক্ষমতা কনটেইনারের হাতে। তাৎক্ষণিক: কনটেইনার থামান, হোস্ট পরীক্ষা করুন। ভবিষ্যতে: এমন mount নিষিদ্ধ করুন।
20. **২০:** ঘাটতি: restart policy নেই, healthcheck নেই, monitoring/alerting নেই। সমাধান: `restart: unless-stopped`, healthcheck, `docker stats`/log alert ও uptime check।

কেস স্টাডি ১–১০ — মূল শিক্ষা (সংক্ষেপ)

1. **Docker socket:** Socket মাউন্ট = daemon নিয়ন্ত্রণ = হোস্ট root। কখনো অ্যাপ কনটেইনারে মাউন্ট করবেন না।
2. **Secret ফাঁস:** Secret image-এ লিখলে history-তে থেকে যায়; rotate/revoke ছাড়া সমাধান হয় না।
3. **পুরনো base image:** নিয়মিত আপডেট ও CI scan ছাড়া জানা CVE প্রোডাকশনে থেকে যায়।
4. **Privileged:** ডিবাগিংয়ের সুবিধার জন্য প্রোডাকশনে privileged চালানো হোস্ট দখলের পথ খুলে দেয়।
5. **Latest tag:** Mutable tag-এ rollback ব্যর্থ হয়; immutable tag/digest ও deploy রেকর্ড দরকার।
6. **Down -v:** প্রোডাকশনে ভলিউম মুছে ফেলা ডেটা হারানোর সরাসরি কারণ; ব্যাকআপ ও নিষেধাজ্ঞা দরকার।
7. **লগে ডিস্ক পূর্ণ:** Log rotation না থাকলে লগই হোস্ট অচল করে দেয়।
8. **Root কনটেইনার:** Non-root user ও read-only filesystem ছাড়া ছোট দুর্বলতাও persistence-এ পরিণত হয়।
9. **DNS/নেটওয়ার্ক:** IP hardcode করবেন না; সার্ভিস নাম ও একই network ব্যবহার করুন।
10. **Resource limit:** Limit না থাকলে একটি কনটেইনার পুরো হোস্টের সার্ভিস ধ্বংস করতে পারে।

চূড়ান্ত পরীক্ষা — মার্কিং গাইড

সেকশন	পূর্ণ নম্বর দেওয়ার শর্ত
A — তত্ত্ব (প্রতি প্রশ্ন ৫)	সঠিক সংজ্ঞা (২), পার্থক্য/কারণ ব্যাখ্যা (২), TechNova উদাহরণ (১)। শুধু সংজ্ঞা দিলে সর্বোচ্চ ২।
B — কনফিগারেশন (প্রতি প্রশ্ন ১০)	কাজ করে এমন কোড/YAML (৫), প্রতিটি সিদ্ধান্তের ব্যাখ্যা (৩), নিরাপত্তা ও সেরা অভ্যাস (২)। `version:` key থাকলে ১ নম্বর কাটা; আসল secret থাকলে ওই প্রশ্নে ০।
C — হাতে-কলমে (প্রতি কাজ ৫)	কাজ সম্পন্ন ও আউটপুট দেখানো (৩), আউটপুটের ব্যাখ্যা (২)। শুধু কমান্ড লিখলে ১।
D — ইনসিডেন্ট (১৫)	Triage (৩), নির্ণয় ধাপ (৪), কারণ ও সমাধান (৪), যোগাযোগ (২), post-incident পরিবর্তন (২)।

ক্যাপস্টোন মূল্যায়ন রুব্রিক

ক্যাপস্টোন: TechNova MERN প্ল্যাটফর্মের সম্পূর্ণ কনটেইনারাইজেশন — web (Next.js), api (Express), mongo (MongoDB) — Compose-এ চালু, registry-তে push, প্রোডাকশন-প্রস্তুত কনফিগারেশনসহ। মোট ১০০ নম্বর।

মানদণ্ড	নম্বর	উত্তীর্ণ (পূর্ণ নম্বরের কাছাকাছি) দেখতে কেমন
Multi-stage Dockerfile (web ও api)	২০	Cache-বান্ধব ক্রম, ছোট base image, non-root user, healthcheck, exec-form CMD, `.dockerignore` — দুটি সার্ভিসেই।
Compose স্ট্যাক	২০	তিনটি সার্ভিস একই network-এ, named volume-এ MongoDB ডেটা, healthcheck-ভিত্তিক depends_on, restart policy, resource limit, `version:` key নেই।
নিরাপত্তা	২০	কোনো secret image/Git-এ নেই, read-only filesystem যেখানে সম্ভব, capability drop, Trivy scan পরিষ্কার বা ব্যাখ্যাসহ ব্যবস্থা নেওয়া।
কার্যকারিতা প্রমাণ	১৫	`docker compose up -d`-এর পর web → api → mongo পুরো প্রবাহ কাজ করে; লগ ও curl আউটপুট সংরক্ষিত।
Deployment ও rollback	১৫	Immutable tag/digest-এ push, deploy ধাপ নথিভুক্ত, rollback পরীক্ষা করা হয়েছে ও তার প্রমাণ আছে।
ডকুমেন্টেশন ও ব্যাকআপ	১০	README-তে চালানো, থামানো, ব্যাকআপ ও restore ধাপ লেখা; ব্যাকআপ থেকে restore পরীক্ষা করা হয়েছে।

গ্রেড: ৯০–১০০ অসাধারণ; ৭৫–৮৯ ভালো; ৬০–৭৪ উত্তীর্ণ; ৬০-এর নিচে পুনরায় জমা দিতে হবে — নিরাপত্তা মানদণ্ডে ১০-এর নিচে পেলে স্বয়ংক্রিয়ভাবে পুনরায় জমা।